Agatha Barbosa Marinho dos Santos
Bruna Kinjo Luiz Pinto
Kayke Ahrens Biscegli

Classificação Automática de Chamados de TI Utilizando Processamento de Linguagem Natural com Explicabilidade Baseada em Grafos

São Paulo
2026

Agatha Barbosa Marinho dos Santos
Bruna Kinjo Luiz Pinto
Kayke Ahrens Biscegli

Classificação Automática de Chamados de TI Utilizando Processamento de Linguagem Natural com Explicabilidade Baseada em Grafos

Trabalho de Conclusão de Curso II apresentado ao Senac Santo Amaro como requisito para conclusão do curso de Bacharelado em Ciência da Computação.

Senac – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Curso de Bacharelado em Ciência da Computação

Orientador: Prof. Douglas José Baquião Ribeiro

São Paulo
2026

Resumo

Este trabalho investiga como se comparam o SHAP e o LIME quanto à fidelidade e à estabilidade das explicações de um classificador BERTimbau, e de que forma um grafo de co-ocorrência contribui para tornar a classificação auditável, em chamados de suporte de TI em português brasileiro. A motivação é que classificadores baseados em *transformers* não revelam por que atribuíram uma categoria a um chamado, o que dificulta a auditoria de suas decisões. Os dados reúnem chamados anonimizados cedidos pela Bayer e pela Aché, num total de 53.279 chamados em 11 categorias, divididos em treino, validação e teste de modo que textos repetidos fiquem no mesmo conjunto. Foram comparados quatro cenários, que acrescentam um componente por vez: um *baseline* SVM com TF-IDF, o *fine-tuning* do BERTimbau, as explicações SHAP e LIME, e o grafo de co-ocorrência ponderado por essas explicações. O BERTimbau obteve F1-macro de 0,628 e acurácia de 0,724 no conjunto de teste.

Palavras-chave: Processamento de Linguagem Natural. BERT. Inteligência Artificial Explicável. Grafos de Co-ocorrência. Classificação de Tickets de TI.

Lista de ilustrações

Figura 1 – Exemplo de explicação gerada pelo LIME 25
Figura 2 – Representação dos valores SHAP por *feature* 27
Figura 3 – Protótipo de baixa fidelidade da interface por Agatha Barbosa 44
Figura 4 – Protótipo de baixa fidelidade da interface por Bruna Kinjo 44
Figura 5 – Protótipo de baixa fidelidade da interface por Kayke Ahrens 45
Figura 6 – Protótipo de baixa fidelidade da interface final - Tela do grafo 45
Figura 7 – Protótipo de baixa fidelidade da interface final - Tela das estatísticas . 46
Figura 8 – Protótipo de média fidelidade - visualização do grafo de co-ocorrência . 47
Figura 9 – Protótipo de média fidelidade - visualização estatística dos *tokens* . . . 47
Figura 10 – Cronograma de atividades – Diagrama de Gantt 59
Figura 11 – Autorização de uso de dados concedida pela Aché Laboratórios Farmacêuticos 62
Figura 12 – Autorização de uso de dados concedida pela Bayer S.A. 64

Lista de tabelas

Tabela 1 – Comparação entre trabalhos relacionados 38
Tabela 2 – Cenários experimentais para avaliação do sistema 40
Tabela 3 – Bibliotecas utilizadas em cada etapa do projeto 41

Lista de abreviaturas e siglas

AOPC	*Area Over the Perturbation Curve* (Área sobre a Curva de Perturbação)
API	*Application Programming Interface* (Interface de Programação de Aplicações)
BEGNN	*BERT-Enhanced Graph Neural Network*
BERT	*Bidirectional Encoder Representations from Transformers*
BERTimbau	Variante do BERT pré-treinada para o português brasileiro
BiLSTM	*Bidirectional Long Short-Term Memory*
CNN	*Convolutional Neural Network* (Rede Neural Convolucional)
CPU	*Central Processing Unit* (Unidade Central de Processamento)
CSS	*Cascading Style Sheets* (Folhas de Estilo em Cascata)
GCN	*Graph Convolutional Network* (Rede Neural Convolucional em Grafos)
GloVe	*Global Vectors for Word Representation*
GNN	*Graph Neural Network* (Rede Neural de Grafos)
GPT	*Generative Pre-trained Transformer*
GPU	*Graphics Processing Unit* (Unidade de Processamento Gráfico)
HOOM	*Hybrid Online Offline Model*
ITSM	*IT Service Management* (Gerenciamento de Serviços de TI)
JSON	*JavaScript Object Notation*
LGPD	Lei Geral de Proteção de Dados Pessoais
LIME	*Local Interpretable Model-Agnostic Explanations*
LOO	*Leave-One-Out* (Deixar Um de Fora)
LSTM	*Long Short-Term Memory*
mBERT	*Multilingual BERT*
MLM	*Masked Language Modeling*

MLP	*Multilayer Perceptron* (Perceptron Multicamadas)
NER	*Named Entity Recognition* (Reconhecimento de Entidades Nomeadas)
NLTK	*Natural Language Toolkit*
NSP	*Next Sentence Prediction*
PLN	Processamento de Linguagem Natural
PMI	*Pointwise Mutual Information* (Informação Mútua Pontual)
PT-BR	Português Brasileiro
RNN	*Recurrent Neural Network* (Rede Neural Recorrente)
RoBERTa	*Robustly Optimized BERT Pretraining Approach*
SHAP	*SHapley Additive exPlanations*
SLA	*Service Level Agreement* (Acordo de Nível de Serviço)
SMOTE	*Synthetic Minority Oversampling Technique*
SVM	*Support Vector Machine* (Máquina de Vetores de Suporte)
TF-IDF	*Term Frequency–Inverse Document Frequency*
TI	Tecnologia da Informação
XAI	*Explainable Artificial Intelligence* (Inteligência Artificial Explicável)
XLM-RoBERTa	*Cross-lingual Language Model RoBERTa*
ZHAW	*Zürcher Hochschule für Angewandte Wissenschaften* (Universidade de Ciências Aplicadas de Zurique)

Sumário

1 INTRODUÇÃO 11
1.1 Contextualização do problema 11
1.2 Formulação do problema 12
1.3 Justificativa 12
1.4 Objetivo 13
1.4.1 Objetivo principal 13
1.4.2 Objetivos secundários 13

2 REVISÃO BIBLIOGRÁFICA 14
2.1 Sistemas de *Help Desk* e a Triagem de Chamados de TI 14
2.1.1 Definição e importância dos sistemas de *help desk* nas organizações 14
2.1.2 O processo manual de triagem e seus problemas 14
2.1.3 Abordagens de aprendizado de máquina para classificação automática de *tickets* 15
2.1.3.1 Modelos clássicos — *Naive Bayes* e SVM com TF-IDF 15
2.1.3.2 Redes neurais convolucionais aplicadas a *tickets* 15
2.1.3.3 Classificação de chamados no contexto brasileiro 16
2.1.4 Limitações dos modelos existentes 17
2.1.4.1 O debate BERT versus SVM em domínios especializados 17
2.1.4.2 A caixa-preta e a verificação das decisões em produção 17
2.2 Processamento de Linguagem Natural para Classificação de Texto . 18
2.2.1 Representações vetoriais de texto 18
2.2.1.1 *TF-IDF* e modelos baseados em frequência 18
2.2.1.2 *Embeddings* estáticos — *Word2Vec* e *GloVe* 18
2.2.1.3 Limitações das representações estáticas — polissemia e contexto 19
2.3 O Modelo BERT e suas Variantes 19
2.3.1 Arquitetura *Transformer* e mecanismo de atenção 19
2.3.1.1 *Self-attention* e atenção bidirecional 19
2.3.1.2 Pré-treinamento — *Masked Language Modeling* e *Next Sentence Prediction* . . 20
2.3.1.3 *Fine-tuning* para tarefas *downstream* 21
2.3.2 BERT 21
2.3.2.1 Arquitetura e configurações *BERT-base* e *BERT-large* 21
2.3.2.2 *Token* [CLS] e classificação de sequências 21
2.3.3 BERTimbau — BERT para o português brasileiro 22
2.3.3.1 Corpus de pré-treinamento e diferenças em relação ao mBERT 22

2.3.3.2 Desempenho em tarefas de PLN em PT-BR . . . 22
2.3.4 *Fine-tuning* de BERT para classificação de chamados de TI . . . 23
2.4 Inteligência Artificial Explicável (XAI) . . . 23
2.4.1 Definição e motivação . . . 23
2.4.1.1 O problema da caixa-preta em aprendizado profundo . . . 23
2.4.1.2 Confiança, explicabilidade e adoção em produção . . . 24
2.4.1.3 Taxonomia de métodos XAI — intrínsecos versus *post-hoc*, locais versus globais 24
2.4.2 LIME — *Local Interpretable Model-Agnostic Explanations* . . . 24
2.4.2.1 Funcionamento — perturbações e modelo local interpretável . . . 24
2.4.2.2 Aplicação em classificação de texto . . . 25
2.4.2.3 Vantagens e limitações . . . 26
2.4.3 SHAP — *SHapley Additive exPlanations* . . . 26
2.4.3.1 Fundamentos — valores de Shapley da teoria dos jogos . . . 26
2.4.3.2 Propriedades matemáticas — eficiência, simetria e ausência . . . 27
2.4.3.3 *KernelSHAP* e aplicação em modelos de texto . . . 27
2.4.4 Comparação entre SHAP e LIME . . . 28
2.4.4.1 Consistência e estabilidade das explicações . . . 28
2.4.4.2 Custo computacional . . . 28
2.4.4.3 Justificativa para uso conjunto neste trabalho . . . 28
2.4.5 XAI aplicado a classificação de texto e tickets de TI . . . 29
2.4.5.1 LIME em classificação multiclasse de texto . . . 29
2.4.5.2 SHAP e LIME em ambiente ITSM corporativo . . . 29
2.5 Grafos de Co-ocorrência como Representação Estrutural de Texto . 30
2.5.1 Fundamentos de grafos aplicados a texto . . . 30
2.5.1.1 Definição de grafo de co-ocorrência — nós, arestas e janela deslizante . . . 30
2.5.1.2 Ponderação de arestas — frequência, PMI . . . 30
2.5.2 *TextGCN* — formalização do grafo de co-ocorrência . . . 31
2.5.2.1 Grafo heterogêneo documento-palavra . . . 31
2.5.2.2 Redes neurais de grafos e propagação de informação . . . 31
2.5.3 Combinação de BERT com grafos de co-ocorrência . . . 32
2.5.3.1 *Embeddings* BERT como *features* iniciais dos nós . . . 32
2.5.3.2 Resultados em vocabulário técnico especializado . . . 32
2.5.4 Grafo de co-ocorrência combinado com XAI . . . 32
2.5.4.1 Ponderação de nós por *scores* de explicabilidade . . . 32
2.5.4.2 Visualização interativa como camada de auditabilidade . . . 33
2.6 Síntese da Literatura e Posicionamento do Trabalho . . . 33
2.6.1 Lacunas identificadas no estado da arte . . . 33
2.6.1.1 Ausência de XAI em sistemas de classificação de tickets em produção . . . 33
2.6.1.2 Carência de soluções robustas para PT-BR em *helpdesk* . . . 33

2.6.1.3 Custo-benefício do BERT sem camada de explicabilidade . . . 34
2.6.2 Direcionamento para a proposta . . . 34
2.7 Metodologia da pesquisa bibliográfica . . . 34
2.8 Fundamentos . . . 34
2.8.1 Processamento de Linguagem Natural . . . 34
2.8.2 Modelos de Linguagem Baseados em *Transformer* . . . 35
2.8.3 Inteligência Artificial Explicável . . . 35
2.8.4 Grafo de Co-ocorrência . . . 35
2.8.5 Auditabilidade . . . 36
2.8.6 Sistema de *Help Desk* e Triagem de Chamados . . . 36
2.9 Trabalhos relacionados . . . 36
2.9.1 Trabalho 1: Text Classification of Service Desk Tickets . . . 37
2.9.2 Trabalho 2: A Hybrid Continual Machine Learning Model for Efficient Hierarchical Classification of Domain-Specific Text in The Presence of Class Overlap (Case Study: IT Support Tickets) . . . 37
2.9.3 Trabalho 3: Classificação hierárquica de textos aplicado ao contexto chamados de suporte . . . 37

3 METODOLOGIA . . . 39
3.1 Tipo de pesquisa e hipótese . . . 39
3.1.1 Hipótese de pesquisa . . . 39
3.2 Procedimentos da abordagem proposta . . . 39
3.2.1 Cenários experimentais . . . 40
3.3 Ferramentas, tecnologias e ambientes . . . 40
3.4 Procedimentos experimentais e métricas . . . 41
3.4.1 Métricas de desempenho do classificador . . . 41
3.4.2 Critérios de validação . . . 42

4 DESENVOLVIMENTO . . . 43
4.1 Modelagem e arquitetura . . . 43
4.1.1 Visão geral da solução . . . 43
4.1.2 Protótipos da interface . . . 43
4.1.3 Arquitetura da solução . . . 47
4.2 Detalhes de implementação . . . 49
4.2.1 Tecnologias e ferramentas . . . 49
4.2.2 Funcionalidades principais . . . 49
4.2.3 Metodologia de implementação . . . 50
4.3 Experimentos e cenários de teste . . . 50
4.3.1 Cenários A e B: classificação . . . 50
4.3.2 Cenário C: explicações SHAP e LIME . . . 50

4.3.3 Cenário D: grafo de co-ocorrência 51
4.4 Coleta e tratamento de dados 51
4.4.1 Origem e autorização de uso dos dados 51
4.4.2 *Dataset* e *fine-tuning* 51

5 RESULTADOS E ANÁLISE 53
5.1 Apresentação dos resultados 53
5.1.1 Desempenho da classificação (cenários A e B) 53
5.1.2 Explicabilidade: SHAP e LIME (cenário C) 53
5.1.3 Grafo de co-ocorrência (cenário D) 53
5.2 Análise crítica e comparação 53
5.3 Discussão 53

6 CONCLUSÃO 54
6.1 Retomada do problema e dos objetivos 54
6.2 Contribuições para a Computação 54
6.3 Limitações e trabalhos futuros 54

REFERÊNCIAS 55

APÊNDICES 58

APÊNDICE A – CRONOGRAMA DE ATIVIDADES 59

ANEXOS 60

ANEXO A – AUTORIZAÇÃO DE USO DE DADOS — ACHÉ LABORATÓRIOS FARMACÊUTICOS 61

ANEXO B – AUTORIZAÇÃO DE USO DE DADOS — BAYER S.A. 63

1 Introdução

A gestão eficiente de *tickets* de suporte técnico em organizações de TI enfrenta desafios crescentes devido ao volume elevado de chamados e à complexidade semântica dos textos gerados por usuários não técnicos (ZEMP, 2021; RAZMA; JURKUS, 2026). Este trabalho investiga o uso de um *pipeline* de classificação automática utilizando Processamento de Linguagem Natural (PLN) com modelos BERT (*Bidirectional Encoder Representations from Transformers*, modelos de linguagem baseados em redes neurais profundas), técnicas de explicabilidade SHAP (*SHapley Additive exPlanations*) e LIME (*Local Interpretable Model-Agnostic Explanations*) — métodos que identificam quais termos do texto influenciaram a decisão do modelo — e visualização em grafos de co-ocorrência (estruturas que representam relações entre termos do texto) para tornar transparente o processo decisório.

1.1 Contextualização do problema

Os *helpdesks* corporativos processam diariamente milhares de *tickets* com descrições variadas que misturam jargão técnico, erros ortográficos e expressões coloquiais (ZEMP, 2021). Isso sobrecarrega os analistas, tornando a triagem manual lenta, inconsistente e propensa a erros.

Estudos recentes demonstram que sistemas automatizados com redes neurais profundas alcançam até 92% de acurácia em classificação multi-categoria (ZEMP, 2021), mas sua natureza de "caixa-preta", modelos que produzem predições sem revelar os critérios utilizados para tomá-las, pode gerar resistência entre equipes operacionais que precisam confiar nas decisões algorítmicas para roteamento, o que torna a explicabilidade relevante nesses sistemas (RAZMA; JURKUS, 2026). Em sistemas reais, o bom desempenho não garante a adoção: no estudo de Zemp (2021), a equipe de suporte recusou a função que separava requisições de incidentes, mesmo com 93% de acurácia de validação (Seção 2.1.4.2).

A raiz matemática dessa intransparência está no mecanismo central desses modelos. A cada camada de processamento, o modelo realiza operações do tipo:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

onde Q, K e V são matrizes de consulta, chave e valor derivadas da entrada (VASWANI et al., 2017). Essas transformações se repetem em centenas de milhões de parâmetros distribuídos em múltiplas camadas, tornando impossível rastrear por que uma palavra específica influenciou determinada classificação. Embora o modelo produza resultados precisos, o caminho percorrido até a decisão permanece não auditável, ou seja, não é possível rastrear nem reproduzir a explicação de uma predição individual, propriedade

definida em detalhe na Seção 2.8.5. Isso justifica a necessidade de técnicas de explicabilidade como SHAP e LIME para atribuir a importância de cada termo na predição (LUNDBERG; LEE, 2017; RIBEIRO; SINGH; GUESTRIN, 2016).

No contexto brasileiro, onde grande parte da comunicação técnica ocorre em português, modelos tradicionais como o *Naive Bayes*, que classifica textos com base em probabilidades, e redes MLP (*Multilayer Perceptron*), que são redes neurais simples sem capacidade de entender o contexto das palavras, apresentam limitações significativas ao lidar com *tickets* reais, pois não conseguem capturar a variabilidade semântica nem o vocabulário técnico específico do domínio de TI (YOKOTA, 2025).

1.2 Formulação do problema

Classificadores baseados em *transformers* alcançam bom desempenho na triagem de chamados de TI, mas não revelam por que atribuíram uma categoria a um chamado. Existem métodos que explicam essas decisões, como SHAP e LIME, mas ainda é preciso verificar se, para chamados em português brasileiro, essas explicações, apresentadas em forma de grafo, permitem a um analista acompanhar o caminho até a decisão.

Diante disso, este trabalho busca responder à seguinte pergunta: **em chamados de suporte de TI em português brasileiro, como se comparam o SHAP e o LIME quanto à fidelidade e à estabilidade das explicações de um classificador BERTimbau, e de que forma um grafo de co-ocorrência contribui para tornar o processo de classificação auditável?**

1.3 Justificativa

A triagem de chamados em sistemas de *Help Desk* ainda é frequentemente realizada manualmente por analistas de suporte, que precisam interpretar descrições textuais de problemas e direcionar os chamados para a equipe técnica responsável. Esse processo pode ser lento, sujeito a inconsistências e dependente da experiência individual dos analistas.

Técnicas de Processamento de Linguagem Natural (PLN) e aprendizado de máquina têm sido utilizadas para automatizar essa tarefa por meio da classificação automática de *tickets*. No entanto, muitos desses modelos funcionam como caixas-pretas, dificultando a compreensão de como as decisões são tomadas.

A área de *Explainable Artificial Intelligence* (XAI) busca justamente aumentar a transparência desses sistemas, permitindo compreender quais fatores influenciam as decisões dos modelos. Nesse contexto, a utilização de modelagem em grafos pode oferecer uma forma visual e estrutural de representar relações entre termos e evidenciar como essas relações contribuem para a classificação.

Dessa forma, investigar se mecanismos de explicabilidade baseados em grafos podem tornar o processo decisório mais compreensível representa uma contribuição relevante tanto para aplicações práticas de suporte técnico quanto para o estudo de transparência em sistemas de inteligência artificial.

1.4 Objetivo

Este trabalho estrutura-se a partir de um objetivo central, que define o escopo principal da pesquisa, e de objetivos específicos que representam as metas intermediárias e os subprodutos analíticos necessários para responder à questão central de investigação.

1.4.1 Objetivo principal

Investigar como uma camada de explicabilidade (SHAP e LIME) e grafos de co-ocorrência, acrescentada a um *pipeline* de classificação baseado em *transformers*, contribui para a auditabilidade do processo decisório, comparando a qualidade das explicações dos dois métodos, utilizando chamados de suporte de TI em português brasileiro.

1.4.2 Objetivos secundários

1. Analisar o ganho de desempenho do modelo BERTimbau com *fine-tuning* em relação ao *baseline* SVM com TF-IDF, utilizando a métrica F1-macro.

2. Comparar a explicabilidade gerada por SHAP e LIME em termos de fidelidade, estabilidade, concordância entre os métodos e relevância semântica das explicações produzidas para chamados de suporte técnico em português brasileiro.

3. Analisar se a visualização em grafos de co-ocorrência ponderados por valores SHAP e LIME fornece auditabilidade adicional ao processo de classificação, no sentido de rastrear cada decisão até os termos, a janela de palavras e os valores SHAP e LIME que a originaram, em relação à exibição isolada de *scores* de importância.

4. Validar a consistência semântica das explicações geradas pelo *pipeline* no contexto de chamados de suporte técnico em português brasileiro, verificando se os termos destacados correspondem ao vocabulário técnico esperado para cada categoria classificada.

2 Revisão Bibliográfica

Esta seção apresenta os fundamentos conceituais que sustentam a proposta deste trabalho. Os temas foram organizados em seis blocos para apresentar a evolução do problema até chegar às limitações identificadas na literatura e à proposta deste trabalho: (1) Sistemas de *Help Desk* e a triagem de chamados de TI; (2) Processamento de Linguagem Natural para classificação de texto; (3) o modelo BERT e suas variantes; (4) Inteligência Artificial Explicável (XAI); (5) grafos de co-ocorrência como representação estrutural de texto; e (6) a síntese da literatura e posicionamento do trabalho.

2.1 Sistemas de *Help Desk* e a Triagem de Chamados de TI

2.1.1 Definição e importância dos sistemas de *help desk* nas organizações

Sistemas de *help desk*, também denominados sistemas de suporte técnico, centrais de serviço ou plataformas ITSM (*IT Service Management*), são estruturas organizacionais e tecnológicas responsáveis por centralizar e gerenciar as solicitações de suporte submetidas pelos usuários de uma organização. Esses sistemas recebem chamados relacionados a falhas de infraestrutura, problemas em aplicações, solicitações de acesso, dúvidas operacionais e demais ocorrências que interrompem ou dificultam o trabalho dos colaboradores (ZEMP, 2021).

A importância desses sistemas para a continuidade operacional das organizações é amplamente reconhecida na literatura. Razma e Jurkus (2026) destacam que empresas modernas geram milhares de chamados de suporte mensalmente, e que cada chamado deve ser corretamente categorizado e encaminhado à equipe responsável para que o prazo de resolução estabelecido nos acordos de nível de serviço (SLA, do inglês *Service Level Agreement*) seja cumprido. O não cumprimento desses prazos impacta diretamente a produtividade organizacional e a satisfação dos usuários.

2.1.2 O processo manual de triagem e seus problemas

Em grande parte das organizações, a triagem de chamados ainda é realizada manualmente por analistas de suporte, que leem a descrição textual do problema submetida pelo usuário, determinam a categoria correta e encaminham o chamado à equipe técnica responsável (ZEMP, 2021). Esse processo apresenta limitações estruturais que se tornam mais evidentes à medida que o volume de chamados cresce.

A inconsistência na categorização é um dos principais problemas. Analistas diferentes podem categorizar o mesmo tipo de chamado de formas distintas, e um mesmo analista

pode ser inconsistente ao longo do tempo. Além disso, a triagem manual depende do conhecimento individual de cada analista sobre a estrutura de categorias da organização, conhecimento que não é facilmente transferível e que se perde quando há rotatividade de pessoal (WAHBA, 2023).

Chamados categorizados incorretamente são encaminhados para equipes que não possuem competência para resolvê-los, isso gera um ciclo de reatribuição de chamados que eleva o tempo de resolução, desperdiça o tempo da equipe e deixa o usuário frustrado (HARUN; HUSPI; IAHAD, 2023). Pereira et al. (2023) observam que, no contexto brasileiro, esse problema é agravado pela informalidade da linguagem utilizada pelos usuários ao descrever seus problemas, o que dificulta ainda mais a categorização manual.

2.1.3 Abordagens de aprendizado de máquina para classificação automática de *tickets*

A automação da triagem de chamados por meio de aprendizado de máquina tem sido investigada de forma crescente na literatura. O problema é tratado como uma tarefa de classificação supervisionada de texto: dado o histórico de chamados já categorizados, treina-se um modelo capaz de predizer a categoria de novos chamados a partir de sua descrição textual.

2.1.3.1 Modelos clássicos — *Naive Bayes* e SVM com TF-IDF

As primeiras abordagens automatizadas para classificação de chamados utilizaram algoritmos clássicos de aprendizado de máquina combinados com representações baseadas em frequência de termos. Harun, Huspi e Iahad (2023) investigaram a classificação automática de perguntas em um sistema de *help desk* universitário, comparando *Naive Bayes* e SVM (*Support Vector Machine*), um algoritmo que classifica textos encontrando a melhor separação possível entre as categorias, com vetorização TF-IDF. Os resultados demonstraram que o SVM com TF-IDF supera o *Naive Bayes* em acurácia e F1-score (métrica que combina precisão e revocação em um único valor, detalhada na Seção 3.4.1), estabelecendo-se como referência para abordagens clássicas nesse domínio.

Apesar de sua eficiência computacional e facilidade de interpretação, esses modelos apresentam limitações na captura de relações semânticas entre termos. A representação TF-IDF trata cada palavra de forma independente, sem considerar o contexto em que ela aparece, o que restringe a capacidade do modelo de distinguir categorias semanticamente próximas.

2.1.3.2 Redes neurais convolucionais aplicadas a *tickets*

Com o avanço das técnicas de aprendizado profundo, modelos baseados em redes neurais convolucionais (CNN) passaram a ser aplicados à classificação de chamados de

suporte. Paramesh e Shreedhara (2022) propuseram um classificador baseado em CNN com *word embeddings* (representações vetoriais densas de palavras aprendidas por redes neurais) para categorização automática de *tickets* de *service desk* de infraestrutura de TI. Os autores compararam o modelo proposto com SVM, *Naive Bayes*, Regressão Logística e *K-Nearest Neighbor*, demonstrando que a CNN supera todos os modelos clássicos em acurácia e F1-score, especialmente em categorias com descrições textuais mais longas e complexas.

A vantagem das CNN em relação aos modelos clássicos reside na capacidade de identificar automaticamente quais grupos de palavras que aparecem juntas são importantes para a classificação, sem que um especialista precise definir isso manualmente. Essa progressão de modelos lineares com TF-IDF para redes profundas com *embeddings*, estabelece o contexto histórico sobre o qual os modelos baseados em *transformers* representam um novo salto qualitativo.

2.1.3.3 Classificação de chamados no contexto brasileiro

A literatura sobre classificação automática de chamados de suporte em português brasileiro ainda é escassa, e os trabalhos existentes evidenciam limitações importantes das abordagens mais simples nesse contexto específico.

Pereira et al. (2023) apresentaram um modelo de classificação automática de chamados de suporte de TI em sete categorias, atingindo 75% de precisão média com modelos de aprendizado de máquina tradicionais. Os autores disponibilizaram publicamente o código, o modelo e os dados anonimizados, e desenvolveram um protótipo *web* com *backend* e *frontend* para uso dos analistas de TI, tornando-o um dos poucos trabalhos nacionais com *dataset* público e interface funcional. No entanto, a acurácia de 75% deixa margem considerável para melhoria, especialmente em cenários com maior número de categorias ou com classes semanticamente próximas.

Yokota (2025) investigou a classificação hierárquica de chamados de suporte utilizando *Naive Bayes* e *Multilayer Perceptron* (MLP) com duas variações arquiteturais. Os resultados, avaliados por F1-macro para lidar com o desbalanceamento entre classes, foram insatisfatórios, evidenciando que modelos simples falham em capturar as nuances semânticas presentes em textos técnicos de suporte em PT-BR. A autora conclui que a abordagem proposta revelou desafios substanciais na classificação automática de textos especializados, evidenciando a necessidade de abordagens metodológicas mais sofisticadas, o que reforça a escolha do BERTimbau neste trabalho.

2.1.4 Limitações dos modelos existentes

2.1.4.1 O debate BERT versus SVM em domínios especializados

Wahba (2023), em sua tese de doutorado na Universidade de Western Ontario, propõe um contraponto relevante ao uso indiscriminado de modelos baseados em *transformers* para classificação de texto em domínios especializados. Com base em quatro anos de estudos empíricos com dados reais de uma empresa parceira, a autora demonstra que chamados de suporte de TI utilizam vocabulário técnico específico e de baixa ambiguidade. Nesse contexto, a principal vantagem do BERT, que é compreender palavras com múltiplos sentidos dependendo do contexto, tem impacto reduzido, tornando o SVM com TF-IDF competitivo em termos de desempenho.

Com base nessa observação, Wahba propõe o modelo HOOM (*Hybrid Online Offline Model*), que combina um classificador SVM linear treinado em modo *offline* com um classificador *Passive Aggressive Classifier*. Esse segundo classificador é um algoritmo de aprendizado incremental que atualiza o modelo a cada nova amostra, mantendo-se "passivo" quando a predição está correta e "agressivo" ao corrigir erros e treinado em modo *online* para detectar mudanças na distribuição dos dados ao longo do tempo. O modelo é descrito como mais eficiente, interpretável e reproduzível do que abordagens baseadas em *transformers*, que são modelos de linguagem baseados em redes neurais profundas e que serão detalhados na Seção 2.3.

Este argumento é diretamente relevante para o presente trabalho, pois os cenários A (SVM com TF-IDF) e B (BERTimbau) permitem medir, nos chamados analisados, o ganho do modelo mais complexo (Seção 3.2.1). Além do desempenho, investiga-se se a camada de explicabilidade traz um benefício de auditabilidade ao classificador.

2.1.4.2 A caixa-preta e a verificação das decisões em produção

Zemp (2021), em estudo desenvolvido na ZHAW com dados reais de um banco privado de médio porte com atuação internacional, treinou três tipos de modelos: CNN, BiLSTM, que é uma rede neural recorrente bidirecional capaz de processar sequências de texto nos dois sentidos, e RoBERTa, uma variante aprimorada do BERT com treinamento mais robusto. O estudo utilizou mais de um milhão de *tickets* históricos, atingindo, com a CNN, 92% de acurácia de validação e F1-macro de 0,75 em mais de 300 classes. O sistema foi parcialmente implantado em produção, e os agentes de suporte foram convidados a utilizá-lo em seu trabalho diário.

Na avaliação com a equipe de suporte, a previsão do serviço afetado foi considerada útil por unanimidade e mantida no sistema. Já a classificação entre requisições e incidentes, embora tenha atingido 93% de acurácia de validação, foi recusada: a equipe a considerou potencialmente perigosa, por poder levar a avaliações superficiais dos chamados, e entendeu que essa distinção deve ser aprendida pelos próprios analistas no trabalho.

Em um ambiente ITSM corporativo multilíngue com mais de 87 mil *tickets*, Razma e Jurkus (2026) aplicaram LIME e SHAP para gerar explicações das predições do modelo de regressão logística e, no caso do LIME, também do mBERT (multilingual BERT), uma versão do BERT treinada simultaneamente em mais de 100 idiomas, e afirmam que essas explicações contribuem para a interpretabilidade e a confiança em sistemas de automação de ITSM multilíngues.

Esses estudos ajudam a justificar a proposta deste trabalho: sistemas de classificação de chamados de TI em produção real funcionam, mas a ausência de mecanismos de explicabilidade limita sua auditabilidade. A proposta deste TCC busca lidar com essa limitação.

2.2 Processamento de Linguagem Natural para Classificação de Texto

2.2.1 Representações vetoriais de texto

2.2.1.1 *TF-IDF* e modelos baseados em frequência

A representação mais simples de um documento para fins de classificação é o modelo *bag-of-words*: cada documento é representado como um vetor de contagens de palavras, sem considerar a ordem em que aparecem. O *TF-IDF* é um refinamento desse modelo que pondera cada termo pelo produto de sua frequência no documento e pelo inverso de sua frequência no corpus, reduzindo o peso de termos muito comuns e aumentando o peso de termos mais informativos (RAZMA; JURKUS, 2026).

Apesar de sua simplicidade, o *TF-IDF* ainda é amplamente utilizado como *baseline* em tarefas de classificação de texto, especialmente quando combinado com SVM, e apresenta desempenho competitivo em domínios com vocabulário técnico específico e baixa polissemia, ou seja, onde as palavras raramente mudam de sentido dependendo do contexto (WAHBA, 2023). Sua principal limitação é a incapacidade de capturar relações semânticas entre termos: palavras sinônimas ou semanticamente relacionadas são tratadas como completamente distintas.

2.2.1.2 *Embeddings* estáticos — *Word2Vec* e *GloVe*

Os *embeddings* de palavras representam um avanço significativo em relação às abordagens baseadas em frequência, ao mapear cada palavra para um vetor denso em um espaço de baixa dimensionalidade, de forma que palavras semanticamente relacionadas sejam representadas por vetores próximos. O *Word2Vec*, proposto por Mikolov et al. (2013), aprende *embeddings* a partir de grandes corpora (plural de *corpus*, conjunto amplo de

textos usado para treinamento) textuais utilizando redes neurais rasas, treinadas para predizer palavras a partir de seu contexto ou contexto a partir de palavras.

O GloVe (*Global Vectors for Word Representation*) utiliza estatísticas globais de co-ocorrência do corpus para produzir representações com propriedades algébricas notáveis: vetores de palavras semanticamente relacionadas tendem a ser próximos no espaço vetorial, e operações aritméticas entre vetores refletem relações semânticas — o exemplo clássico é que o vetor de "rei" menos "homem" mais "mulher" resulta em um vetor próximo ao de "rainha".

Esses *embeddings* estáticos representaram um avanço expressivo na qualidade das representações textuais e foram amplamente adotados como camada de entrada em modelos de aprendizado profundo para PLN. No entanto, eles atribuem um único vetor a cada palavra, independentemente do contexto em que ela aparece, limitação conhecida como o problema da representação estática.

2.2.1.3 Limitações das representações estáticas — polissemia e contexto

O grande problema dos *embeddings* estáticos é que eles não conseguem identificar quando uma palavra muda de sentido dependendo do contexto — como "banco", que pode ser um assento ou uma instituição financeira. Em textos técnicos de suporte de TI, isso se manifesta em termos como "acesso", "serviço" ou "sistema", que podem ter significados muito distintos dependendo do chamado.

Essa limitação motivou o desenvolvimento de representações contextualizadas, nas quais o *embedding* de cada palavra é computado dinamicamente a partir do contexto completo da frase. Os modelos baseados na arquitetura *Transformer*, em particular o BERT, representam a abordagem dominante nessa direção (DEVLIN et al., 2019).

2.3 O Modelo BERT e suas Variantes

2.3.1 Arquitetura *Transformer* e mecanismo de atenção

2.3.1.1 *Self-attention* e atenção bidirecional

A arquitetura *Transformer* é composta por dois blocos principais: um *encoder* (codificador) e um *decoder* (decodificador), cada um formado por camadas empilhadas (VASWANI et al., 2017). O *encoder* é responsável por receber a sequência de entrada e transformá-la em representações vetoriais ricas em informação contextual. O *decoder*, por sua vez, utiliza essas representações para gerar a sequência de saída como uma tradução ou resposta. Cada camada do *encoder* é composta por dois sub-componentes principais: o mecanismo de *self-attention*, que permite que cada *token*, ou seja, cada unidade mínima de texto em que o modelo divide a entrada, "observe" todos os outros da sequência, e uma rede neural densa aplicada posição a posição (*feed-forward network*). O resultado de cada sub-componente

passa por uma operação de normalização (*layer normalization*) que estabiliza o treinamento (VASWANI et al., 2017). Vale destacar que modelos como o BERT utilizam apenas o bloco *encoder*, pois seu objetivo é produzir representações contextuais do texto de entrada, e não gerar novas sequências (DEVLIN et al., 2019).

A arquitetura *Transformer*, proposta por Vaswani et al. (2017), introduziu o mecanismo de *self-attention* como alternativa às redes recorrentes para processamento de sequências. As RNN (*Recurrent Neural Networks*) são uma classe de redes neurais projetadas para processar dados sequenciais, nos quais o contexto e a ordem temporal importam, processando os *tokens* um por vez, da esquerda para a direita. As LSTM (*Long Short-Term Memory*) são um tipo especializado de RNN projetado para aprender dependências de longo prazo em sequências, mitigando o problema de desvanecimento do gradiente presente nas RNN simples. Apesar dessas melhorias, ambas as arquiteturas ainda processam a sequência de forma estritamente sequencial, o que limita o paralelismo computacional e dificulta a captura de relações entre termos distantes.

Para compreender o mecanismo de atenção, considere a frase "o banco bloqueou meu acesso": para determinar o significado da palavra "banco", é necessário considerar as demais palavras da frase — especialmente "bloqueou" e "acesso" — que indicam tratar-se de uma instituição financeira e não de um assento. O *self-attention* formaliza esse processo: para cada *token* da sequência, o mecanismo calcula um score de relevância em relação a todos os outros *tokens*, indicando o quanto cada um deve "influenciar" a representação do *token* atual. "Atender simultaneamente a todas as posições" significa que esse cálculo é realizado em paralelo para todos os pares de *tokens* da sequência, possibilitando a captura de dependências de longo alcance de forma muito mais eficiente do que as arquiteturas recorrentes.

Modelos como o GPT utilizam atenção unidirecional (cada *token* só pode atender aos *tokens* anteriores). O BERT, por sua vez, utiliza atenção bidirecional: cada *token* atende simultaneamente a todos os outros *tokens* da sequência, tanto os que aparecem antes quanto os que aparecem depois. Essa bidirecionalidade é fundamental para a qualidade das representações contextualizadas geradas pelo modelo, uma vez que o significado de uma palavra frequentemente depende tanto de seu contexto anterior quanto posterior (DEVLIN et al., 2019).

2.3.1.2 Pré-treinamento — *Masked Language Modeling* e *Next Sentence Prediction*

O BERT é pré-treinado em dois objetivos complementares sobre grandes corpora textuais não rotulados. O primeiro, *Masked Language Modeling* (MLM), mascara aleatoriamente 15% dos *tokens* de cada sequência de entrada e treina o modelo para predizer os *tokens* mascarados a partir do contexto bidirecional restante. Esse objetivo força o modelo a aprender representações ricas que capturam o significado de cada palavra em seu contexto completo (DEVLIN et al., 2019).

O segundo objetivo, *Next Sentence Prediction* (NSP), treina o modelo a determinar se duas sentenças apresentadas em sequência são consecutivas no texto original ou foram pareadas aleatoriamente. Esse objetivo permite ao modelo aprender relações semânticas entre sentenças, o que é útil para tarefas como inferência textual e resposta a perguntas.

2.3.1.3 *Fine-tuning* para tarefas *downstream*

Após o pré-treinamento, o BERT pode ser adaptado para tarefas específicas por meio do processo de *fine-tuning*: o modelo pré-treinado é inicializado com os pesos aprendidos no pré-treinamento, uma camada de saída específica para a tarefa é adicionada ao topo da arquitetura, e o modelo completo é treinado por um número reduzido de épocas sobre o conjunto de dados rotulado da tarefa-alvo (DEVLIN et al., 2019).

Essa abordagem de pré-treinamento seguido de *fine-tuning* é particularmente eficaz porque o modelo já adquiriu conhecimento linguístico geral durante o pré-treinamento, e o *fine-tuning* apenas adapta esse conhecimento para o domínio e a tarefa específicos. Para classificação de chamados de TI, o *fine-tuning* ajusta os pesos do modelo para que as representações produzidas sejam especialmente discriminativas entre as categorias de chamados presentes no *dataset*.

2.3.2 BERT

2.3.2.1 Arquitetura e configurações *BERT-base* e *BERT-large*

Devlin et al. (2019) propuseram o BERT em duas configurações. O *BERT-base* possui 12 camadas de *encoder Transformer*, 12 cabeças de atenção e 110 milhões de parâmetros. O *BERT-large* possui 24 camadas, 16 cabeças de atenção e 340 milhões de parâmetros. Ambas as versões foram pré-treinadas sobre o corpus da Wikipedia em inglês e o BookCorpus, totalizando aproximadamente 3,3 bilhões de *tokens*.

O BERT estabeleceu o estado da arte em onze tarefas de PLN no momento de sua publicação, incluindo classificação de texto, reconhecimento de entidades nomeadas, resposta a perguntas e inferência textual. Seu lançamento popularizou o uso de modelos pré-treinados em tarefas de PLN, consolidando a abordagem de pré-treinamento em larga escala seguido de *fine-tuning* como a estratégia dominante para tarefas de PLN.

2.3.2.2 *Token* [CLS] e classificação de sequências

Para tarefas de classificação de sequências, o BERT utiliza um *token* especial [CLS] (abreviação de *Classification*, marcador inserido pelo modelo para agregar a representação global da sequência) inserido no início de cada sequência de entrada. Após o processamento pelas camadas de *encoder*, a representação vetorial correspondente ao [CLS] agrega informação contextual de toda a sequência e é usada como entrada para a camada de classificação (DEVLIN et al., 2019). Durante o *fine-tuning*, essa camada de classificação,

tipicamente uma camada *linear* seguida de *softmax*, aprende a mapear a representação do [CLS] para as probabilidades de cada categoria. Neste trabalho, essa representação é usada para classificar o chamado de TI em uma das categorias predefinidas, e os valores SHAP e LIME são calculados sobre os tokens de entrada para identificar quais contribuíram mais para a predição.

2.3.3 BERTimbau — BERT para o português brasileiro

2.3.3.1 Corpus de pré-treinamento e diferenças em relação ao mBERT

Souza, Nogueira e Lotufo (2020) desenvolveram o BERTimbau, modelo BERT pré-treinado especificamente para o português brasileiro, a partir do BrWaC (*Brazilian Web as Corpus*), um grande corpus de páginas *web* em português com 2,68 bilhões de *tokens*. Duas versões foram disponibilizadas: BERTimbau-base (12 camadas, 110M parâmetros) e BERTimbau-large (24 camadas, 335M parâmetros).

A principal diferença em relação ao mBERT está no foco linguístico: o mBERT foi pré-treinado sobre 104 idiomas simultaneamente, o que dilui a capacidade de representação para cada língua individualmente. O BERTimbau, ao ser pré-treinado exclusivamente sobre texto em português brasileiro, aprende padrões morfológicos, sintáticos e semânticos específicos do idioma com maior profundidade.

2.3.3.2 Desempenho em tarefas de PLN em PT-BR

Souza, Nogueira e Lotufo (2020) avaliaram o BERTimbau em três tarefas de PLN em português: reconhecimento de entidades nomeadas (NER), similaridade textual entre sentenças e inferência textual (*Recognizing Textual Entailment*). O modelo obteve desempenho de estado da arte nas três tarefas, consolidando-se como referência para aplicações de PLN em PT-BR.

Para o presente trabalho, a escolha do BERTimbau em detrimento do mBERT é motivada por dois fatores. Primeiro, os chamados de suporte analisados estão em português brasileiro, idioma para o qual o BERTimbau possui representações mais ricas. Segundo, os chamados utilizam terminologia técnica de TI em português, que pode diferir substancialmente de sua versão em inglês. E o BERTimbau, pré-treinado em corpus nacional, tem maior probabilidade de ter encontrado esse vocabulário durante o pré-treinamento.

O BERTimbau foi mantido como modelo deste trabalho, dado que os chamados analisados estão predominantemente em português brasileiro. A avaliação de modelos multilíngues ou mais recentes fica indicada como trabalho futuro.

2.3.4 *Fine-tuning* de BERT para classificação de chamados de TI

A aplicação de BERT e modelos derivados à classificação de chamados de TI tem demonstrado resultados expressivos na literatura recente. Borges (2024) realizou uma comparação sistemática entre BERT e algoritmos clássicos, SVM e *Naive Bayes*, em tarefas de classificação de texto em português e inglês, utilizando quatro *datasets* públicos. Os resultados confirmaram que o BERT supera consistentemente os modelos tradicionais em acurácia e F1, especialmente em tarefas multiclasse com nuances semânticas mais complexas, embora com custo computacional consideravelmente maior.

Zemp (2021) comparou CNN, BiLSTM e RoBERTa sobre mais de um milhão de *tickets* reais de um banco privado de médio porte, em mais de 300 classes. A melhor CNN atingiu F1-macro de 0,75. Com o *fine-tuning* simples, o RoBERTa ficou abaixo desse valor; somente após um pré-treinamento adicional do modelo de linguagem com os próprios *tickets* ele superou a CNN, por uma margem de 0,018.

Razma e Jurkus (2026) compararam regressão logística com TF-IDF frente a mBERT e XLM-RoBERTa, uma variante do RoBERTa desenvolvida especificamente para lidar com múltiplos idiomas simultaneamente, em um corpus de 87 mil *tickets* multilíngues de uma empresa de manufatura. Os resultados confirmaram que *transformers* superam modelos clássicos em macro-F1, especialmente para classes de baixa frequência e semanticamente complexas. Os autores aplicaram o LIME ao mBERT e à regressão logística, e o SHAP à regressão logística, o que mostra ser viável explicar um BERT *fine-tuned* com LIME sobre chamados reais de ITSM; eles afirmam que as explicações contribuem para a interpretabilidade e a confiança.

2.4 Inteligência Artificial Explicável (XAI)

2.4.1 Definição e motivação

2.4.1.1 O problema da caixa-preta em aprendizado profundo

Modelos de aprendizado profundo, incluindo o BERT, são frequentemente descritos como caixas-pretas: produzem predições com alta acurácia, mas sem revelar quais características da entrada determinaram a saída. Isso acontece porque esses modelos possuem centenas de milhões de parâmetros internos, valores numéricos ajustados durante o treinamento, organizados em múltiplas camadas de cálculos complexos. Essa estrutura torna impossível para um ser humano acompanhar ou explicar, passo a passo, como o modelo chegou a uma determinada decisão (SALIH et al., 2023).

Em contextos como a triagem de chamados de TI, essa opacidade representa um problema prático concreto. Analistas de suporte e gestores precisam auditar as decisões do sistema, identificar erros sistemáticos e eventualmente contestar categorizações incorretas. Um modelo que não fornece justificativa para suas predições é difícil de depurar, difícil de

melhorar e difícil de justificar perante *stakeholders* (partes interessadas, como gestores e diretores) organizacionais (ZEMP, 2021).

2.4.1.2 Confiança, explicabilidade e adoção em produção

No estudo de Zemp (2021), a equipe de suporte recusou parte do sistema de classificação, como discutido na Seção 2.1.4.2. Razma e Jurkus (2026) afirmam que as explicações LIME favorecem a interpretabilidade e a confiança no sistema, pois permitem verificar se os termos destacados como mais influentes fazem sentido para aquela categoria específica, embora não tenham medido a reação dos operadores.

Essa ideia é consistente com a literatura de XAI mais ampla: a explicabilidade não melhora necessariamente a acurácia do modelo, mas aumenta a confiança dos usuários no sistema e facilita a identificação de falhas sistemáticas, o que é essencial para a adoção sustentada em ambientes de produção.

2.4.1.3 Taxonomia de métodos XAI — intrínsecos versus *post-hoc*, locais versus globais

Os métodos de XAI podem ser classificados segundo duas dimensões principais (SALIH et al., 2023). A primeira dimensão distingue métodos intrínsecos dos métodos *post-hoc*. Métodos intrínsecos são aqueles em que a interpretabilidade é uma propriedade do próprio modelo — árvores de decisão, regressão logística e modelos lineares em geral são interpretáveis por *design*, pois seus parâmetros têm significado diretamente interpretável. Métodos *post-hoc* são aplicados após o treinamento, sobre modelos de qualquer complexidade, e produzem explicações sem modificar o modelo original.

A segunda dimensão distingue explicações locais das globais. Explicações locais descrevem o comportamento do modelo para uma instância específica de entrada (exemplo: por que o modelo classificou *este* chamado como "Acesso"?). Explicações globais descrevem o comportamento geral do modelo (exemplo: quais termos são mais importantes para cada categoria em média?). SHAP e LIME são métodos *post-hoc* que operam primariamente no nível local, embora o SHAP permita agregações que produzem explanações globais.

2.4.2 LIME — *Local Interpretable Model-Agnostic Explanations*

2.4.2.1 Funcionamento — perturbações e modelo local interpretável

LIME foi proposto por Ribeiro, Singh e Guestrin (2016) como um método de explicação local que funciona com qualquer tipo de modelo e qualquer tipo de entrada, sem precisar conhecer sua estrutura interna. O método parte do princípio de que, embora um modelo de aprendizado profundo seja globalmente complexo e não-linear, seu comportamento em torno de um exemplo específico pode ser aproximado por um modelo simples e interpretável.

O processo de geração de uma explicação LIME para uma instância de texto ocorre em quatro etapas. Primeiro, o texto original é perturbado pela remoção ou substituição

aleatória de palavras, criando um conjunto de versões modificadas da frase. Segundo, cada versão modificada é submetida ao modelo para que ele produza uma predição, permitindo observar se e como a resposta muda conforme as palavras são alteradas. Terceiro, as versões modificadas são ponderadas pela similaridade com o texto original, sendo que versões mais próximas do original recebem peso maior. Quarto, um modelo linear simples é ajustado sobre esse conjunto, e seus coeficientes indicam quais palavras mais influenciaram a classificação (RIBEIRO; SINGH; GUESTRIN, 2016).

Formalmente, o LIME resolve o seguinte problema de otimização para encontrar a explicação $\xi(x)$ de uma instância x:

$$\xi(x) = \arg\min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g) \tag{2.1}$$

onde f é o modelo de caixa-preta a ser explicado, G é o conjunto de modelos interpretáveis candidatos, $\mathcal{L}$ é uma função de perda que mede a fidelidade da explicação g ao modelo f na vizinhança de x, π_x é uma função de proximidade que pondera as instâncias perturbadas pela similaridade com x, e $\Omega(g)$ é uma medida de complexidade do modelo explicativo (RIBEIRO; SINGH; GUESTRIN, 2016). O objetivo é encontrar o modelo interpretável mais simples possível que seja localmente fiel ao comportamento do modelo complexo.

A Figura 1 ilustra esse processo: as palavras destacadas indicam sua contribuição positiva ou negativa para a classe predita.

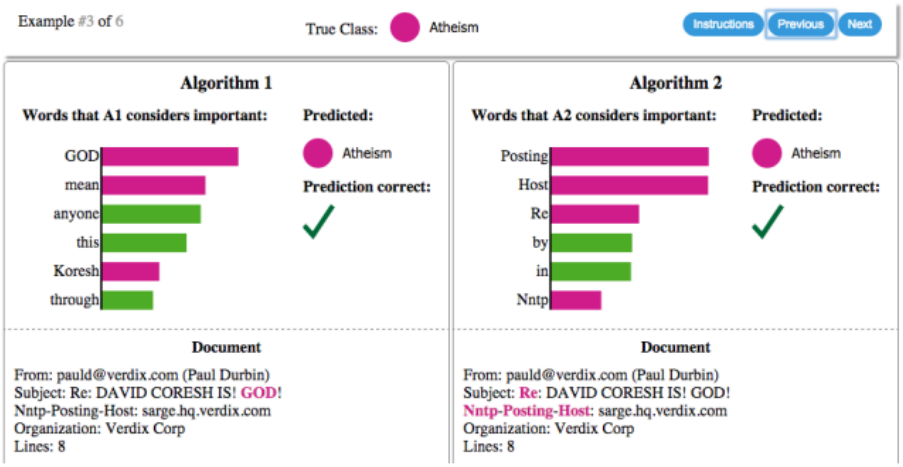


Figura 1 – Exemplo de explicação gerada pelo LIME

Fonte: Ribeiro, Singh e Guestrin (2016)

2.4.2.2 Aplicação em classificação de texto

No contexto de classificação de texto, cada palavra (ou *token*) da entrada é tratada como uma *feature*, ou seja, uma característica binária que indica se está presente ou

ausente no texto. A explicação LIME indica quais palavras, quando presentes, aumentam a probabilidade de cada classe, e quais palavras a diminuem. O resultado pode ser visualizado como um destaque colorido sobre o texto original, onde palavras em verde contribuem positivamente para a classe predita e palavras em vermelho contribuem negativamente (MUSTAFA; Hama Saeed, 2025).

Mustafa e Hama Saeed (2025) demonstraram que a aplicação de LIME na classificação de texto não apenas produz explicações úteis para o usuário final, mas também contribui para melhorar o processo de pré-processamento: ao revelar que *stopwords* recebem pesos desproporcionalmente altos nas explicações, o LIME evidenciou a necessidade de sua remoção.

2.4.2.3 Vantagens e limitações

A principal vantagem do LIME é sua generalidade: o método funciona com qualquer modelo e qualquer tipo de entrada sem necessidade de acesso ao interior do modelo. Isso o torna especialmente adequado para modelos de caixa-preta como o BERT, cujos mecanismos internos são de difícil interpretação direta.

Sua principal limitação é a instabilidade das explicações. Por depender de amostragem aleatória de perturbações, execuções diferentes do LIME sobre a mesma instância podem produzir explicações distintas, especialmente quando há colinearidade entre as *features*, ou seja, quando algumas características extraídas do texto são muito parecidas ou carregam informações semelhantes (SALIH et al., 2023). Para mitigar essa instabilidade, recomenda-se executar o LIME múltiplas vezes e analisar a consistência das explicações, ou fixar a semente aleatória para garantir reprodutibilidade.

2.4.3 SHAP — *SHapley Additive exPlanations*

2.4.3.1 Fundamentos — valores de Shapley da teoria dos jogos

SHAP foi proposto por Lundberg e Lee (2017) com base nos valores de Shapley da teoria dos jogos cooperativos. O problema que o SHAP resolve é: dada uma predição específica de um modelo, como distribuir o "crédito" por essa predição de forma justa entre as *features* de entrada?

Na teoria dos jogos, o valor de Shapley de um jogador em um jogo cooperativo é calculado como a média ponderada de quanto cada jogador contribuiu individualmente em todas as combinações possíveis com os demais jogadores. Lundberg e Lee (2017) adaptam esse conceito para o contexto de modelos de aprendizado de máquina: cada *feature* é um "jogador", a predição do modelo é o "valor do jogo", e o valor SHAP de cada *feature* mede o quanto ela contribuiu para a predição, considerando todas as possíveis ordens em que as *features* poderiam ser incluídas.

Formalmente, o valor de Shapley de uma *feature* i é calculado como:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} \left[f_{S \cup \{i\}}(x_{S \cup \{i\}}) - f_S(x_S) \right] \tag{2.2}$$

onde F é o conjunto de todas as *features*, S é um subconjunto que não inclui i, $f(S)$ é a predição do modelo usando apenas as *features* em S, e $f(S \cup \{i\})$ é a predição incluindo a *feature* i (LUNDBERG; LEE, 2017). O fator $\frac{|S|!(|F|-|S|-1)!}{|F|!}$ garante que todas as ordenações possíveis de inclusão das *features* sejam consideradas com peso igual, assegurando a propriedade de simetria.

A Figura 2 ilustra esse conceito: cada *feature* desloca a predição do valor base até o valor final do modelo.

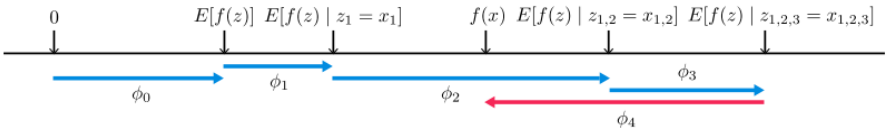


Figura 2 – Representação dos valores SHAP por *feature*

Fonte: Lundberg e Lee (2017)

2.4.3.2 Propriedades matemáticas — eficiência, simetria e ausência

A formulação baseada em valores de Shapley garante ao SHAP três propriedades matemáticas desejáveis que o distinguem de métodos *ad hoc* de atribuição de importância (LUNDBERG; LEE, 2017). A propriedade de eficiência estabelece que a soma dos valores SHAP de todas as *features* iguala a diferença entre a predição do modelo para a instância em questão e o valor base (predição média do modelo sobre o conjunto de dados). A propriedade de simetria estabelece que *features* com contribuições iguais ao modelo recebem valores SHAP iguais, independentemente de sua posição na entrada. A propriedade de ausência estabelece que *features* que não afetam a predição recebem valor SHAP zero.

Essas propriedades conferem ao SHAP uma fundamentação teórica sólida que o LIME não possui, tornando suas explicações mais consistentes e matematicamente justificáveis.

2.4.3.3 *KernelSHAP* e aplicação em modelos de texto

Calcular o valor exato de Shapley se torna inviável quando o modelo possui muitas *features*, pois seria necessário testar todas as combinações possíveis entre elas, o que cresce exponencialmente. Para resolver isso, o *KernelSHAP* (LUNDBERG; LEE, 2017) estima esses valores testando apenas uma amostra dessas combinações, tornando o método viável mesmo em modelos de texto, onde cada *token* é tratado como uma *feature*.

Para modelos baseados em *transformers*, o *KernelSHAP* é a variante mais utilizada. O resultado é um vetor de importância por *token* que pode ser visualizado como mapa de calor sobre o texto de entrada, *tokens* com valores SHAP positivos contribuem para a classe

predita, e *tokens* com valores negativos contribuem contra ela. O presente trabalho pretende utilizar esses valores SHAP por *token* como pesos dos nós no grafo de co-ocorrência, em conjunto com os valores LIME, permitindo comparação visual entre os dois métodos de explicabilidade.

2.4.4 Comparação entre SHAP e LIME

2.4.4.1 Consistência e estabilidade das explicações

Salih et al. (2023) realizaram uma análise comparativa abrangente entre SHAP e LIME, avaliando suas propriedades teóricas e comportamento empírico em tarefas de classificação. Em termos de consistência, o SHAP demonstra maior estabilidade: por ser baseado em uma formulação matemática única com garantias teóricas, suas explicações são menos sensíveis à aleatoriedade do processo de amostragem. O LIME, por depender de perturbações aleatórias e de um processo de ajuste local, pode produzir explicações distintas para a mesma instância em execuções diferentes.

Os autores concluem que SHAP é preferível quando a consistência e a fundamentação teórica das explicações são prioritárias, enquanto o LIME pode ser preferível quando a velocidade de geração das explicações é mais importante do que a estabilidade.

2.4.4.2 Custo computacional

Em termos de custo computacional, o LIME é mais eficiente que o *KernelSHAP* em textos longos porque trabalha com um número fixo de perturbações. Já o *KernelSHAP* exige uma amostragem muito maior de combinações de *features* para gerar estimativas precisas, o que eleva o tempo de processamento (SALIH et al., 2023). Para textos curtos, como os chamados de suporte de TI, que tipicamente contêm poucas dezenas de palavras, essa diferença de custo é menos pronunciada.

2.4.4.3 Justificativa para uso conjunto neste trabalho

A decisão de implementar e comparar SHAP e LIME neste trabalho, em vez de escolher apenas um dos métodos, é fundamentada em duas razões. Primeiro, a comparação entre os dois métodos produz evidência empírica sobre a qualidade e consistência das explicações geradas para chamados de TI em PT-BR, contribuição metodológica relevante para a literatura sobre XAI em domínios especializados. Segundo, a construção do grafo de co-ocorrência proposto utiliza os valores SHAP e LIME como pesos dos nós, permitindo avaliar se a escolha do método de XAI afeta a estrutura e a interpretabilidade do grafo resultante.

2.4.5 XAI aplicado a classificação de texto e tickets de TI

2.4.5.1 LIME em classificação multiclasse de texto

Mustafa e Hama Saeed (2025) propuseram um *framework* de classificação de texto que combina PLN e XAI. O sistema inclui pré-processamento com NLTK (*Natural Language Toolkit*), balanceamento de classes com SMOTE (*Synthetic Minority Oversampling Technique*) para lidar com datasets desbalanceados, e validação cruzada estratificada repetida: uma técnica que divide os dados em múltiplos subconjuntos para avaliar o modelo de forma mais robusta, garantindo que todas as classes estejam representadas em cada divisão. O modelo utilizado foi um híbrido de regressão *ridge* com regressão logística. O LIME foi utilizado para validar a explicabilidade das classificações em seis domínios distintos: medicina, esportes, entretenimento, política, tecnologia e negócios. O modelo híbrido atingiu 99% de acurácia após a aplicação do pré-processamento informado pelas explanações LIME.

O trabalho demonstra que a integração de XAI ao *pipeline* de classificação não é apenas uma camada de interpretabilidade adicionada ao final do processo, mas pode também influenciar decisões de pré-processamento e seleção de *features*, melhorando a qualidade do modelo como um todo.

2.4.5.2 SHAP e LIME em ambiente ITSM corporativo

Razma e Jurkus (2026) aplicaram LIME e SHAP em um ambiente ITSM corporativo real, comparando as explicações geradas para o modelo de regressão logística com *TF-IDF* e para o mBERT *fine-tuned*. Os resultados qualitativos das explicações foram significativamente distintos: o modelo linear tendia a destacar palavras-chave literais com alta frequência no corpus, enquanto o BERT destacava termos semanticamente coerentes com o significado do chamado.

Essa diferença qualitativa nas explicações, documentada por Razma e Jurkus (2026), sugere que o BERTimbau, combinado com SHAP e LIME, tende a produzir explicações mais acuradas em termos de importância de *features* e potencialmente mais interpretáveis para analistas de TI sem formação em aprendizado de máquina, hipótese que será verificada empiricamente no presente trabalho, avaliando se os termos destacados fazem sentido para cada categoria de chamado classificada.

2.5 Grafos de Co-ocorrência como Representação Estrutural de Texto

2.5.1 Fundamentos de grafos aplicados a texto

2.5.1.1 Definição de grafo de co-ocorrência — nós, arestas e janela deslizante

Um grafo de co-ocorrência de texto é uma estrutura $G = (V, E, W, f_{\text{shap}}, f_{\text{lime}})$, onde:

- V representa o conjunto de nós, correspondendo aos *tokens* únicos do texto;
- E representa o conjunto de arestas, conectando pares de *tokens* que co-ocorrem dentro de uma janela deslizante de tamanho w;
- W representa os pesos das arestas, determinados pela frequência de co-ocorrência entre os *tokens*;
- f_{shap} representa os pesos dos nós calculados pelo método SHAP, indicando a contribuição de cada *token* para a predição do modelo (LUNDBERG; LEE, 2017);
- f_{lime} representa os pesos dos nós calculados pelo método LIME, permitindo comparação visual entre os dois métodos de explicabilidade (RIBEIRO; SINGH; GUESTRIN, 2016).

A janela deslizante percorre o texto posição por posição; em cada posição, todos os pares de *tokens* dentro da janela recebem uma aresta ou têm o peso de sua aresta existente incrementado (YAO; MAO; LUO, 2019). Essa representação captura informação estrutural sobre o texto que as representações vetoriais não preservam: dois *tokens* que frequentemente aparecem juntos no mesmo contexto estarão conectados por uma aresta de alto peso no grafo, independentemente de sua posição absoluta na sequência. Para chamados de TI, isso significa que termos técnicos que tendem a co-ocorrer em chamados de uma mesma categoria, como "VPN" e "acesso" em chamados de conectividade, estarão estruturalmente próximos no grafo, com pesos de nós proporcionais à sua importância explicativa calculada por SHAP e LIME.

2.5.1.2 Ponderação de arestas — frequência, PMI

O peso das arestas em um grafo de co-ocorrência pode ser determinado por diferentes métricas. A abordagem mais simples utiliza a frequência bruta de co-ocorrência, ou seja, a contagem de quantas vezes o par de *tokens* co-ocorreu dentro da janela deslizante. Métricas mais sofisticadas, como a Informação Mútua Pontual (PMI, do inglês *Pointwise Mutual Information*), normalizam a frequência de co-ocorrência pelas frequências individuais de cada *token*, favorecendo pares que co-ocorrem mais do que seria esperado pelo acaso (YAO; MAO; LUO, 2019).

No presente trabalho, os pesos das arestas serão determinados pelo número de co-ocorrências do par dentro da janela deslizante (frequência bruta, sem normalização), enquanto os pesos dos *nós*, que constituem o elemento central da proposta, serão determinados pelos valores SHAP e LIME calculados para cada *token* pelo modelo BERTimbau, permitindo comparar visualmente a importância atribuída por cada método. Essa distinção entre ponderação de arestas (estrutura relacional) e ponderação de nós (relevância explicativa) é a inovação central da camada de visualização proposta.

2.5.2 *TextGCN* — formalização do grafo de co-ocorrência

2.5.2.1 Grafo heterogêneo documento-palavra

Yao, Mao e Luo (2019) propuseram o *TextGCN* (*Text Graph Convolutional Network*), modelo que constrói um único grafo heterogêneo para todo o corpus, conectando documentos e palavras. As arestas documento-palavra recebem pesos baseados em TF-IDF, refletindo a importância de cada palavra para cada documento. As arestas palavra-palavra recebem pesos baseados em PMI calculada sobre co-ocorrências em janela deslizante de tamanho 20 sobre todo o corpus, refletindo a força de associação semântica entre pares de palavras.

Essa estrutura permite que as redes neurais de grafos (GCN) propaguem informação simultaneamente entre documentos e palavras, capturando relações semânticas globais que modelos baseados em sequências não conseguem representar diretamente.

2.5.2.2 Redes neurais de grafos e propagação de informação

As redes neurais de grafos (GNN, do inglês *Graph Neural Networks*) operam sobre grafos propagando informação entre nós vizinhos por meio de operações de agregação e transformação. Em cada camada da rede, a representação de cada nó é atualizada com base nas representações de seus vizinhos na iteração anterior (YAO; MAO; LUO, 2019). Após múltiplas camadas, cada nó possui uma representação que agrega informação de sua vizinhança local de múltiplos saltos.

No contexto deste trabalho, a relevância do *TextGCN* é fundamentalmente teórica: ele formaliza matematicamente a estrutura do grafo de co-ocorrência e demonstra sua utilidade para classificação de texto. O presente trabalho não irá implementar GCN, o grafo de co-ocorrência será utilizado exclusivamente como estrutura de visualização para as explanações XAI, não como componente do modelo de classificação.

2.5.3 Combinação de BERT com grafos de co-ocorrência

2.5.3.1 *Embeddings* BERT como *features* iniciais dos nós

Yang e Cui (2021) propuseram o BEGNN (*BERT-Enhanced Graph Neural Network*), que combina representações contextualizadas do BERT com grafos de co-ocorrência para classificação de texto. Na arquitetura BEGNN, o BERT gera *embeddings* contextualizados para cada *token* da entrada; esses *embeddings* são utilizados como *features* iniciais dos nós do grafo de co-ocorrência correspondente ao documento. Em seguida, uma GCN propaga informação entre nós vizinhos, produzindo representações enriquecidas pela estrutura do grafo. A representação final do documento é obtida por agregação das representações dos nós.

Essa combinação aborda uma limitação do BERT quando utilizado isoladamente: o mecanismo de atenção do BERT captura relações entre *tokens*, mas essas relações são implícitas nos pesos de atenção e não explicitamente estruturadas como um grafo. O grafo de co-ocorrência fornece uma estrutura explícita que complementa as representações contextualizadas.

2.5.3.2 Resultados em vocabulário técnico especializado

Yang e Cui (2021) avaliaram o BEGNN em múltiplos *datasets* de classificação de texto, demonstrando que a combinação BERT + grafo supera tanto o BERT isolado quanto o *TextGCN*, especialmente em *datasets* com vocabulário técnico especializado. Esse resultado é relevante para o presente trabalho porque chamados de suporte de TI são exatamente o tipo de texto com vocabulário técnico especializado para o qual o BEGNN demonstrou vantagem, sugerindo que a estrutura de grafo de co-ocorrência captura informação complementar às representações do BERTimbau.

2.5.4 Grafo de co-ocorrência combinado com XAI

2.5.4.1 Ponderação de nós por *scores* de explicabilidade

Talebi e Abdolvand (2025) apresentam o trabalho de maior proximidade metodológica com o presente TCC: aplicam RoBERTa combinado com LIME e um grafo de co-ocorrência para detectar *body shaming*, prática de humilhar pessoas com base em sua aparência física, em redes sociais em inglês. No trabalho de Talebi e Abdolvand, o grafo de co-ocorrência é construído com nós representando *tokens* e arestas ponderadas pelos *scores* LIME de cada *token*. Os *tokens* com maior importância LIME produzem arestas de maior peso com seus vizinhos na janela de co-ocorrência.

O presente trabalho adotará uma abordagem estruturalmente análoga, mas com diferenças importantes: utilizará BERTimbau em vez de RoBERTa, aplicará os valores SHAP e LIME como pesos dos nós em vez dos *scores* LIME como pesos de arestas, e o

domínio de aplicação será a classificação de chamados de suporte de TI em português brasileiro. A escolha de ponderar os nós em vez das arestas reflete a intenção de destacar visualmente os *tokens* individualmente mais relevantes para a predição, enquanto as arestas manterão sua função de representar a estrutura de co-ocorrência do texto.

2.5.4.2 Visualização interativa como camada de auditabilidade

A literatura aponta que a combinação de grafos de co-ocorrência com *scores* de explicabilidade pode ser exibida em interfaces interativas que tornam o processo decisório auditável. Nessa abordagem, os nós do grafo são dimensionados proporcionalmente à importância de cada *token* calculada pelo método XAI, e as arestas são ponderadas pela frequência de co-ocorrência, produzindo uma representação visual que combina estrutura relacional do texto com relevância explicativa do modelo (TALEBI; ABDOLVAND, 2025).

Essa camada de visualização tem o potencial de transformar a saída opaca do modelo em uma representação auditável, o que responde à limitação apontada por Zemp (2021) e Razma e Jurkus (2026): sem acesso às razões da decisão, analistas de suporte não conseguem verificar o comportamento do modelo. O presente trabalho avalia a propriedade técnica de auditabilidade da representação proposta, sem medir a adoção efetiva do sistema por usuários. A solução proposta adota essa abordagem, cuja implementação é detalhada no Capítulo 4.

2.6 Síntese da Literatura e Posicionamento do Trabalho

2.6.1 Lacunas identificadas no estado da arte

2.6.1.1 Ausência de XAI em sistemas de classificação de tickets em produção

O trabalho de Zemp (2021) demonstra que sistemas de classificação de chamados de TI com alta acurácia são tecnicamente viáveis e já foram implantados em produção. No entanto, a ausência de mecanismos de explicabilidade limita a auditabilidade desses sistemas. Os agentes de suporte recebem predições sem justificativa, o que dificulta a identificação de erros sistemáticos. Nenhum dos trabalhos de classificação de chamados de TI identificados no *screening* incorpora uma camada de visualização gráfica para explanações XAI.

2.6.1.2 Carência de soluções robustas para PT-BR em *helpdesk*

Os trabalhos de Yokota (2025) e Pereira et al. (2023) evidenciam que o estado da arte em classificação de chamados de suporte em português brasileiro ainda é incipiente. Os melhores resultados reportados com modelos simples (75% de precisão média com ML tradicional e F1-macro insatisfatório com NB e MLP) são inferiores a modelos baseados

em *transformers* já alcançados em inglês. Não foi identificado nenhum trabalho nacional que aplique BERTimbau com XAI a chamados de suporte de TI.

2.6.1.3 Custo-benefício do BERT sem camada de explicabilidade

Wahba (2023) questiona o uso de BERT para classificação de texto em domínios especializados, argumentando que o custo computacional adicional não se justifica diante da pequena vantagem sobre o SVM em textos técnicos com baixa polissemia. Esse argumento permanece válido enquanto o único critério de avaliação for a acurácia. O presente trabalho examina esse argumento medindo o ganho do BERTimbau sobre o SVM e investigando se a camada de XAI e grafo de co-ocorrência acrescenta auditabilidade ao processo de classificação.

2.6.2 Direcionamento para a proposta

Diante das lacunas identificadas: ausência de XAI em sistemas de classificação de chamados em produção, carência de soluções robustas para PT-BR em *helpdesk* de TI, e a questão do custo-benefício do BERT sem explicabilidade, o presente trabalho investiga um *pipeline* que combina BERTimbau, SHAP, LIME e grafo de co-ocorrência ponderado para classificação de chamados em PT-BR com explicabilidade visual. A descrição detalhada da arquitetura, do *dataset*, dos experimentos e da interface que será desenvolvida é apresentada no Capítulo 4.

2.7 Metodologia da pesquisa bibliográfica

A pesquisa bibliográfica foi realizada nas bases Google Scholar, IEEE Xplore, arXiv e SciELO, utilizando as palavras-chave “BERT”, “Grafos”, “XAI”, “PLN”, “Help Desk”, “Chamados de TI”, “SHAP”, “LIME”, “Tickets”, “Support”, “Pipeline”, “Classificação”, “Caixa Preta” e “Knowledge Graph”, entre outras. Foram selecionados artigos publicados entre 2021 e 2026, em português e inglês, que tivessem relação direta com o tema abordado.

2.8 Fundamentos

Esta seção apresenta os conceitos técnicos fundamentais que sustentam a proposta deste trabalho. O objetivo é fornecer ao leitor as definições necessárias para compreender a solução proposta antes de sua descrição detalhada.

2.8.1 Processamento de Linguagem Natural

Processamento de Linguagem Natural (PLN) é o campo da inteligência artificial dedicado ao desenvolvimento de métodos computacionais para compreender, interpretar e

gerar linguagem humana. No contexto deste trabalho, PLN é utilizado para representar chamados de suporte técnico como vetores numéricos que um modelo de aprendizado de máquina possa processar, e para extrair informações semânticas dos textos submetidos pelos usuários (DEVLIN et al., 2019).

2.8.2 Modelos de Linguagem Baseados em *Transformer*

Modelos de linguagem baseados na arquitetura *Transformer* (VASWANI et al., 2017) aprendem representações contextualizadas de texto por meio de mecanismos de atenção que consideram simultaneamente todas as posições de uma sequência de entrada. O BERT (DEVLIN et al., 2019) é o modelo desta família mais amplamente adotado para tarefas de classificação de texto. O BERTimbau (SOUZA; NOGUEIRA; LOTUFO, 2020) é uma variante do BERT pré-treinada especificamente sobre corpora em português brasileiro, sendo o modelo adotado neste trabalho.

2.8.3 Inteligência Artificial Explicável

Inteligência Artificial Explicável (XAI) refere-se ao conjunto de métodos e técnicas que permitem compreender e auditar as decisões de modelos de aprendizado de máquina (SALIH et al., 2023). Dois métodos *post-hoc* serão utilizados neste trabalho:

- **LIME** (RIBEIRO; SINGH; GUESTRIN, 2016): explica predições individuais ajustando um modelo linear simples em torno de perturbações da entrada original, indicando quais palavras mais contribuíram para a classificação.
- **SHAP** (LUNDBERG; LEE, 2017): atribui a cada palavra da entrada um valor de importância calculado com base na teoria dos jogos cooperativos, com garantias matemáticas de consistência e equidade na distribuição de crédito.

2.8.4 Grafo de Co-ocorrência

Um grafo de co-ocorrência é uma estrutura $G = (V, E, W, f_{\text{shap}}, f_{\text{lime}})$ na qual os nós V representam os termos únicos de um texto e as arestas E conectam pares de termos que aparecem juntos dentro de uma janela deslizante de tamanho fixo (YAO; MAO; LUO, 2019). Neste trabalho, essa estrutura será utilizada como camada de visualização das explicações XAI: os pesos dos nós serão determinados pelos valores SHAP e LIME de cada *token*, e os pesos das arestas pela frequência de co-ocorrência, produzindo uma representação visual que combina estrutura relacional do texto com relevância explicativa do modelo.

2.8.5 Auditabilidade

A literatura de XAI não apresenta uma definição única e consolidada de auditabilidade. Por essa razão, adota-se aqui uma definição operacional, construída a partir de propriedades já estabelecidas na literatura: um sistema de classificação é auditável quando permite rastrear, verificar e reproduzir a explicação de cada decisão do modelo. Trata-se, portanto, de uma propriedade técnica do sistema, e não da confiança percebida por quem o utiliza. Essa distinção é relevante porque os estudos de Zemp (2021) e Razma e Jurkus (2026), ao relatarem a reação de analistas de suporte a sistemas de classificação em produção, tratam da confiança percebida, e não da propriedade técnica definida aqui.

A definição se desdobra em três dimensões:

- **Rastreabilidade**: a capacidade de atribuir a decisão do modelo a termos específicos da entrada, obtida por meio dos valores SHAP (LUNDBERG; LEE, 2017) e LIME (RIBEIRO; SINGH; GUESTRIN, 2016);
- **Estabilidade**: a reprodutibilidade da explicação gerada para uma mesma instância ao longo de execuções repetidas, questão especialmente relevante para o LIME, que depende de amostragem aleatória de perturbações (SALIH et al., 2023);
- **Fidelidade**: o grau em que a explicação reflete o processo de decisão efetivamente utilizado pelo modelo (JACOVI; GOLDBERG, 2020), verificável pela variação da confiança preditiva ao remover da entrada os termos apontados como mais relevantes.

A validação da percepção de confiança por parte de gestores ou analistas de TI não é objeto deste trabalho, sendo reconhecida como extensão natural da pesquisa.

2.8.6 Sistema de *Help Desk* e Triagem de Chamados

Um sistema de *help desk* é uma plataforma que centraliza e gerencia solicitações de suporte técnico submetidas pelos usuários de uma organização. A triagem de chamados consiste em classificar cada solicitação em uma categoria predefinida e encaminhá-la à equipe responsável pela resolução. A automação desse processo, substituindo ou auxiliando a triagem manual, é o problema central abordado por este trabalho (ZEMP, 2021).

2.9 Trabalhos relacionados

Esta seção analisa três trabalhos que enfrentaram o desafio de automatizar a triagem de chamados de suporte de TI, servindo de base comparativa para a evolução proposta neste TCC.

2.9.1 Trabalho 1: Text Classification of Service Desk Tickets

Zemp (2021) abordou o problema da triagem automática num ambiente corporativo com mais de um milhão de chamados. Para resolvê-lo, o autor comparou modelos CNN, BiLSTM e RoBERTa. A metodologia envolveu o treino em larga escala num *dataset* real de um banco privado de médio porte. Os resultados mostraram uma acurácia de 92%. No entanto, a classificação automática entre incidentes e requisições foi considerada potencialmente perigosa pela equipe de suporte, devido ao risco de avaliações superficiais dos chamados. O presente trabalho diferencia-se por focar na explicabilidade via SHAP, LIME e grafos, permitindo que os analistas de TI entendam o "porquê" da classificação.

2.9.2 Trabalho 2: A Hybrid Continual Machine Learning Model for Efficient Hierarchical Classification of Domain-Specific Text in The Presence of Class Overlap (Case Study: IT Support Tickets)

Wahba (2023) enfrentou o mesmo desafio de classificação de *tickets*, mas propôs uma abordagem focada em eficiência para domínios especializados. A solução utilizou um modelo híbrido (HOOM) que combina SVM linear com classificadores *online*. A metodologia demonstrou que em textos técnicos, modelos mais simples podem ser competitivos. Os resultados mostraram boa performance com baixo custo computacional. A limitação identificada é que essa abordagem perde as nuances contextuais que modelos baseados em *Transformers* capturam. O presente trabalho pretende adotar o BERTimbau, específico para PT-BR, combinado com visualização de grafos para suprir a falta de contexto de modelos lineares.

2.9.3 Trabalho 3: Classificação hierárquica de textos aplicado ao contexto chamados de suporte

Yokota (2025) investigou a classificação de chamados especificamente em português brasileiro, utilizando modelos clássicos como *Naive Bayes* e MLP. A metodologia focou na estrutura hierárquica das categorias de TI. Os resultados apresentaram F1-macro insatisfatório para uso em produção. A lacuna principal foi a incapacidade de modelos simples lidarem com o jargão técnico do suporte em português. O presente trabalho diferencia-se pelo uso de uma arquitetura de estado da arte (BERTimbau) treinada especificamente no nosso idioma, superando as limitações semânticas dos modelos testados por Yokota.

Tabela 1 – Comparação entre trabalhos relacionados

Critério	**Zemp (2021)**	**Wahba (2023)**	**Yokota (2025)**	**Este trabalho**
Objetivo	Classificação	Eficiência	Classificação PT-BR	Classificação + XAI
Tecnologia	RoBERTa / CNN	SVM / HOOM	*Naive Bayes* / MLP	BERTimbau + Grafos
Explicabilidade	Não	Intrínseca	Limitada	SHAP / LIME / Grafos
Limitação	Caixa-Preta	Baixo Contexto	Baixa Performance	Custo Computacional

Fonte: Elaborado pelos autores (2026)

3 Metodologia

Este capítulo descreve o plano da pesquisa: o tipo de estudo e a hipótese, as etapas da abordagem proposta, as ferramentas e os procedimentos experimentais com suas métricas. Os dados utilizados, os cenários executados e a implementação do sistema são detalhados no Capítulo 4.

3.1 Tipo de pesquisa e hipótese

Trata-se de uma pesquisa aplicada, de caráter empírico e comparativo: o estudo mede o desempenho de modelos de classificação e a qualidade das explicações produzidas sobre chamados reais de suporte de TI, comparando quatro cenários que acrescentam um componente por vez ao *pipeline* (Tabela 2).

3.1.1 Hipótese de pesquisa

A utilização de modelagem em grafos, ponderados pelos valores SHAP e LIME, como camada de explicabilidade visual para o modelo BERT contribui para a auditabilidade do processo de classificação automatizada, permitindo rastrear quais termos do chamado contribuíram para a categoria atribuída pelo modelo.

3.2 Procedimentos da abordagem proposta

A abordagem proposta segue as etapas abaixo, em ordem cronológica:

1. **Preparação dos dados**: anonimização dos chamados cedidos pelas empresas, filtragem dos chamados em português e harmonização das categorias;
2. **Divisão dos dados**: divisão estratificada pela categoria e agrupada pelo texto do chamado em treino, validação e teste, com semente fixa e índices salvos, de modo que os cenários comparados usem os mesmos conjuntos;
3. **Cenário A**: treinamento do *baseline* SVM com TF-IDF;
4. **Cenário B**: *fine-tuning* do BERTimbau;
5. **Cenário C**: cálculo das explicações SHAP e LIME sobre uma amostra de chamados do conjunto de teste;
6. **Cenário D**: construção do grafo de co-ocorrência com os *scores* dessas explicações;

7. **Avaliação**: cálculo das métricas de classificação e das medidas de qualidade das explicações (Seção 3.4.1).

Os dados e a implementação de cada etapa são detalhados no Capítulo 4.

3.2.1 Cenários experimentais

O experimento será estruturado em quatro cenários comparativos, conforme descrito na Tabela 2, com o objetivo de isolar a contribuição de cada componente do *pipeline*:

Tabela 2 – Cenários experimentais para avaliação do sistema

Cenário	**Modelo**	**XAI**	**Objetivo**
A	SVM + TF-IDF	Nenhum	Baseline clássico
B	BERTimbau	Nenhum	Ganho do transformer
C	BERTimbau	SHAP + LIME	Qualidade das explicações
D	BERTimbau	SHAP + LIME + grafo	Auditabilidade visual

Fonte: Elaborado pelos autores

A comparação entre os cenários A e B avalia o ganho de desempenho do BERTimbau sobre o *baseline* clássico, respondendo ao argumento de Wahba (2023) sobre o custo-benefício dos *transformers* em domínios especializados. A comparação entre os cenários C e D examina a hipótese de pesquisa (Seção 3.1.1), verificando se a camada de grafo de co-ocorrência oferece auditabilidade superior à exibição isolada de *scores* de importância, nos termos definidos na Seção 2.8.5.

3.3 Ferramentas, tecnologias e ambientes

O projeto foi desenvolvido em Python. A Tabela 3 resume as bibliotecas utilizadas em cada etapa e o motivo da escolha.

A versão do SHAP utilizada foi a 0.54.1, e a do LIME, a 0.2.0.1.

O BERTimbau (SOUZA; NOGUEIRA; LOTUFO, 2020) foi escolhido por ser um modelo BERT pré-treinado em português brasileiro. O *baseline* SVM com TF-IDF foi escolhido por ser a referência clássica em classificação de texto e permitir medir o ganho do modelo neural.

Os treinos foram executados em placa de vídeo (GPU) NVIDIA GTX. Já a execução do SHAP e do LIME foi feita em CPU, em um computador Apple.

Tabela 3 – Bibliotecas utilizadas em cada etapa do projeto

Etapa	Biblioteca	Finalidade
Preparação dos dados	*pandas*, *openpyxl*, *pyarrow*, *spaCy*, *PyYAML*	Leitura das planilhas, limpeza, anonimização e armazenamento dos chamados
Classificador clássico (cenário A)	*scikit-learn*	TF-IDF e SVM linear, usados como *baseline*
Classificador neural (cenário B)	*PyTorch*, *Transformers*	*Fine-tuning* e inferência do BERTimbau
Explicabilidade (cenários C e D)	*SHAP*, *LIME*	Importância das palavras para a classe prevista
Métricas e análise	*NumPy*, *SciPy*, *scikit-learn*	Métricas de classificação, Jaccard e Spearman
Grafo e interface	*React*, *Vite*, *vis-network*, *Flask*	Exibição do grafo de co-ocorrência e comunicação da interface com o modelo

Fonte: Elaborado pelos autores

3.4 Procedimentos experimentais e métricas

3.4.1 Métricas de desempenho do classificador

O desempenho dos classificadores (cenários A e B) é medido no conjunto de teste, separado do treino e da validação, por meio das métricas abaixo, amplamente utilizadas em classificação de texto.

- **Acurácia (*Accuracy*)** – proporção de classificações corretas sobre o total de instâncias avaliadas.
- **Precisão (*Precision*)** – proporção de classificações corretas entre os casos previstos para uma determinada categoria, medindo a capacidade do modelo de evitar falsos positivos.
- **Revocação (*Recall*)** – capacidade do modelo de identificar corretamente os chamados pertencentes a cada categoria, medindo a capacidade de evitar falsos negativos.
- **F1-score** – média harmônica entre precisão e revocação, expressa pela fórmula $F1 = 2 \cdot \frac{\text{Precisão} \times \text{Revocação}}{\text{Precisão} + \text{Revocação}}$. Utiliza-se a variante **F1-macro**, que calcula o F1-score de cada categoria e tira a média, tratando todas as categorias com o mesmo peso, independentemente do tamanho. Isso é especialmente importante em conjuntos de dados desbalanceados.

- **Matriz de confusão** – representação tabular que detalha os tipos de erros cometidos pelo modelo, permitindo identificar padrões sistemáticos de confusão entre categorias semanticamente próximas.

3.4.2 Critérios de validação

O sucesso da solução é avaliado sob duas perspectivas complementares.

Desempenho da classificação. O critério definido no TCC1 é que o BERTimbau supere o *baseline* SVM com TF-IDF (cenário A) em pelo menos 5 pontos percentuais de F1-macro. A diferença é calculada no conjunto de teste, com intervalo de confiança de 95%, e é reportada como resultado do estudo, seja ela maior ou menor que o critério. Os mesmos cálculos são repetidos no conjunto de validação.

Qualidade das explicações. SHAP e LIME são comparados sobre uma amostra de cerca de 98 chamados do conjunto de teste, entre acertos e erros do modelo, sem textos repetidos. Cada campo (descrição curta e descrição completa) é explicado separadamente, com o outro fixo, e a perturbação é a remoção de palavras. As propriedades avaliadas são:

- **Fidelidade:** mede se as palavras apontadas como importantes realmente influenciam a decisão do modelo. Usam-se a *comprehensiveness*, a *sufficiency* (AOPC) e o Tau-LOO, comparados a uma linha de base aleatória. As medidas seguem a definição de Attanasio et al. (2023), mas foram implementadas pelos autores, pois a biblioteca *ferret* não pôde ser instalada no ambiente utilizado.

- **Estabilidade:** mede se a explicação se mantém ao repetir o método sobre o mesmo chamado. Repete-se cada explicação cinco vezes e calculam-se a similaridade de Jaccard das cinco palavras mais relevantes, a correlação de Spearman e a fração de repetições com a mesma palavra no topo, em chamados com ao menos oito palavras distintas.

- **Concordância entre SHAP e LIME:** mede se os dois métodos apontam as mesmas palavras para o mesmo chamado.

- **Coerência semântica:** análise qualitativa, nos estudos de caso, de se os termos destacados pelo SHAP e pelo LIME correspondem ao vocabulário técnico esperado para a categoria predita.

Grafo de co-ocorrência. A auditabilidade do grafo é avaliada pela sua rastreabilidade: cada nó e cada aresta pode ser levado de volta ao chamado original, à janela de palavras que o gerou e às notas do SHAP e do LIME.

4 Desenvolvimento

Neste capítulo, apresentam-se a modelagem e a arquitetura da solução, os detalhes de implementação, os experimentos realizados e o tratamento dos dados utilizados. O cronograma das atividades está no Apêndice A.

4.1 Modelagem e arquitetura

4.1.1 Visão geral da solução

A solução consiste em um *pipeline* que classifica chamados de suporte técnico e explica cada decisão, utilizando técnicas de Processamento de Linguagem Natural (PLN) e Inteligência Artificial Explicável (XAI). O *pipeline* é o meio para investigar a auditabilidade do processo decisório: por meio dele, compara-se a qualidade das explicações geradas por SHAP e LIME e avalia-se como o grafo de co-ocorrência permite rastrear quais termos do chamado influenciaram a classificação.

Para isso, o *pipeline* utiliza duas técnicas complementares de explicabilidade: o SHAP, que atribui a cada termo do chamado um valor numérico indicando o quanto ele contribuiu para a classificação, e o LIME, que gera explicações locais testando versões modificadas do texto. Os valores SHAP e LIME são utilizados como pesos dos nós em um grafo de co-ocorrência $G = (V, E, W, f_{\text{shap}}, f_{\text{lime}})$, produzindo uma visualização gráfica que combina a estrutura relacional do texto com a relevância explicativa de cada termo segundo ambos os métodos.

A interface é uma prova de conceito do *pipeline*, voltada a gestores e analistas de TI, e não foi avaliada junto a esse público neste trabalho.

4.1.2 Protótipos da interface

Durante a etapa inicial do projeto foram elaborados protótipos de baixa e média fidelidade com o objetivo de validar a organização visual da interface, o fluxo de navegação e a disposição das funcionalidades relacionadas à explicabilidade do modelo.

A etapa de prototipação de baixa fidelidade teve início com a elaboração individual de diferentes propostas de interface por cada integrante do grupo, permitindo que distintas ideias e abordagens fossem exploradas inicialmente. Posteriormente, as principais características de cada proposta foram analisadas e consolidadas em uma versão única do protótipo que passou a servir como base para a continuidade do projeto.

Os protótipos de baixa fidelidade foram utilizados para estruturar os principais componentes da aplicação, definindo a posição dos menus, área de inserção de chamados, painel

de resultados e espaço destinado às visualizações explicativas.

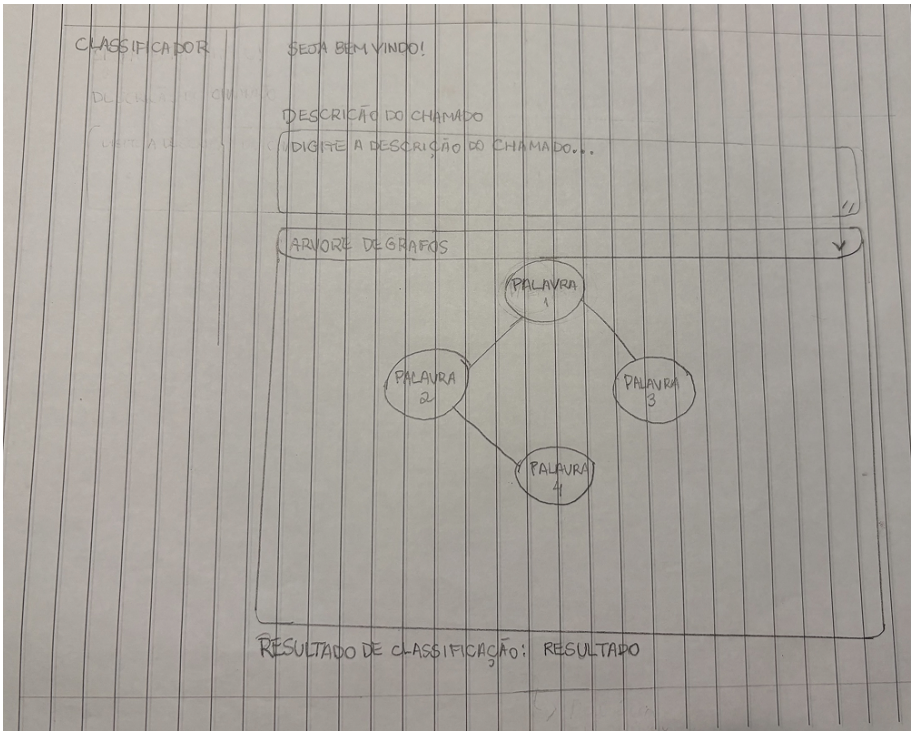


Figura 3 – Protótipo de baixa fidelidade da interface por Agatha Barbosa

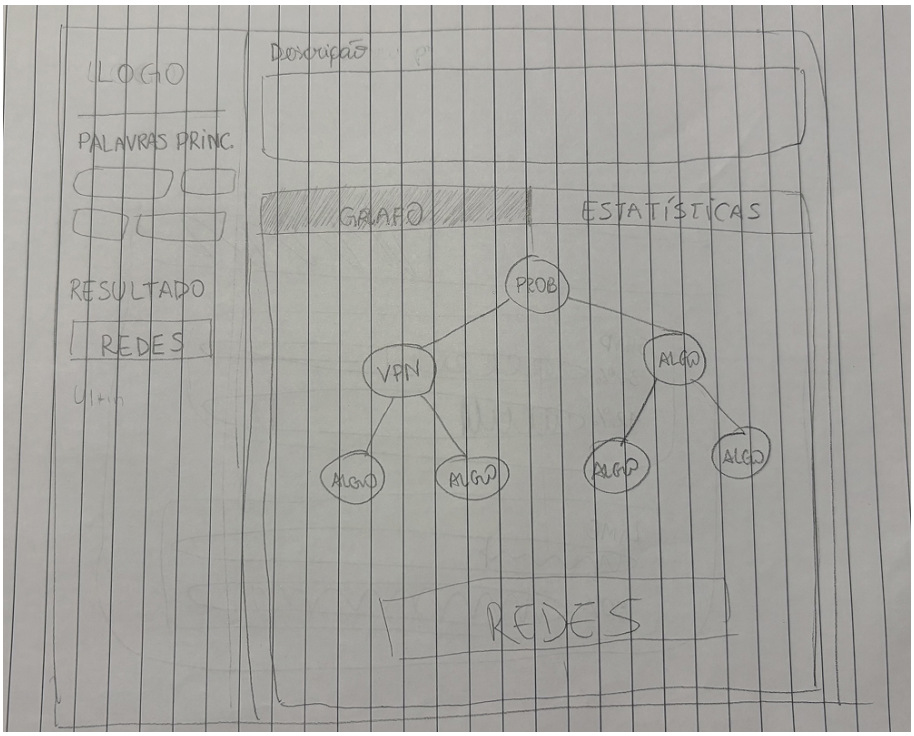


Figura 4 – Protótipo de baixa fidelidade da interface por Bruna Kinjo

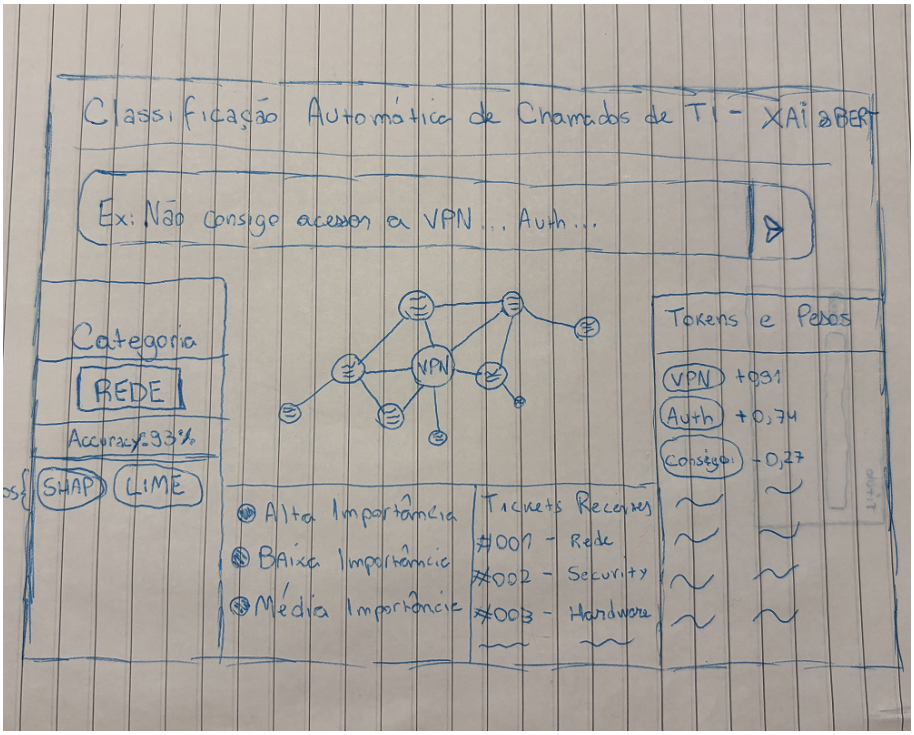


Figura 5 – Protótipo de baixa fidelidade da interface por Kayke Ahrens

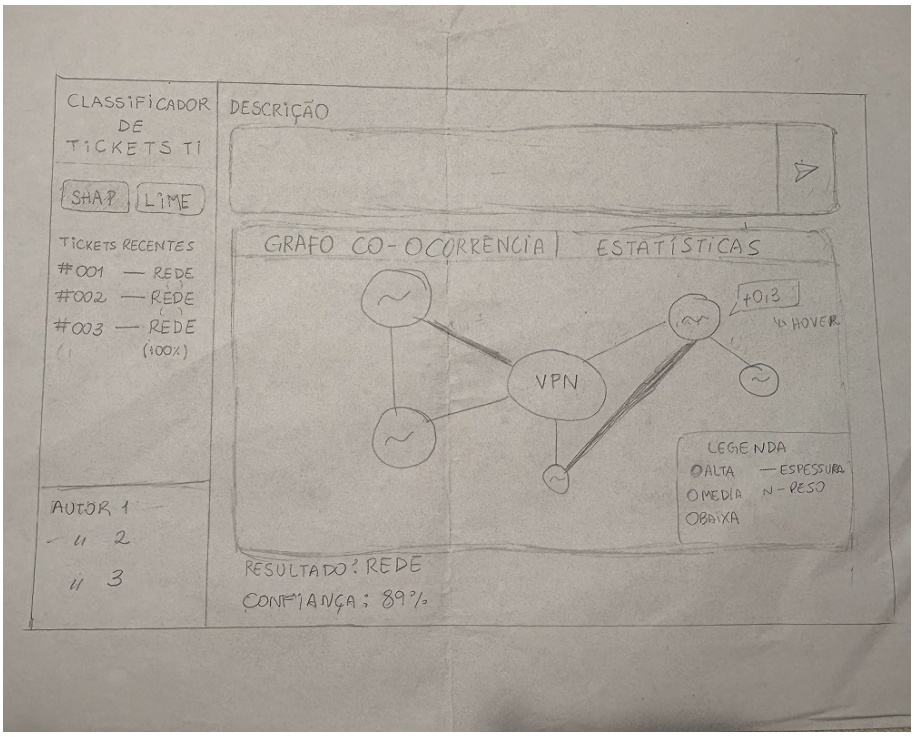


Figura 6 – Protótipo de baixa fidelidade da interface final - Tela do grafo

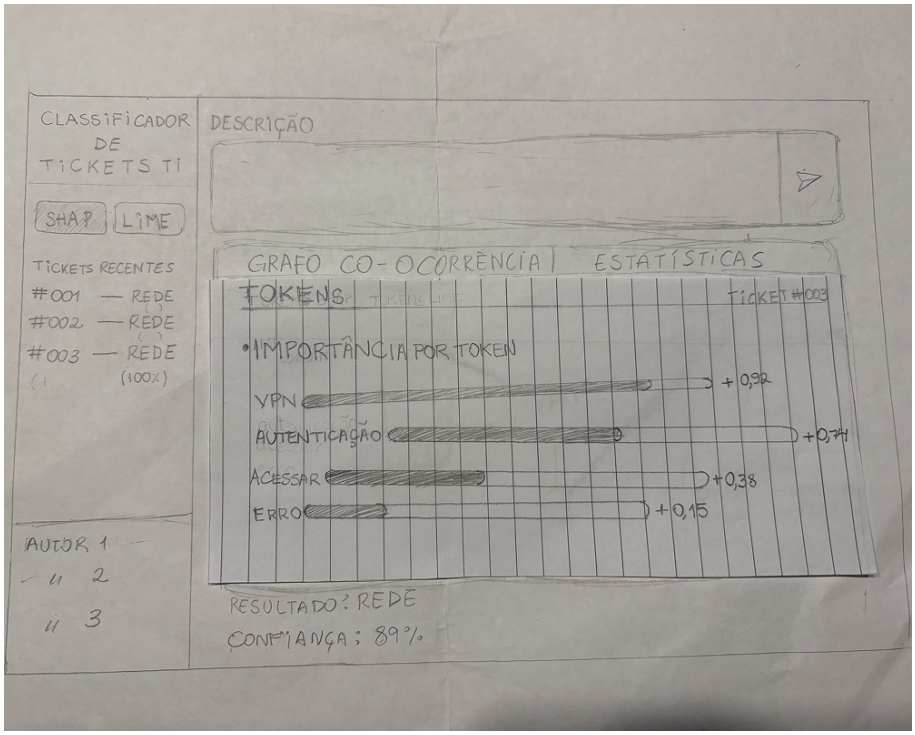


Figura 7 – Protótipo de baixa fidelidade da interface final - Tela das estatísticas

Posteriormente, foram elaborados protótipos de média fidelidade com maior nível de detalhamento visual, com foco em validar como as explicações geradas pelo modelo seriam apresentadas de forma compreensível para os analistas de TI.

A área central do sistema é dedicada às explicações da classificação, organizada em duas abas principais, ambas voltadas para tornar o processo decisório do modelo auditável:

- **Grafo de Co-Ocorrência**: representação visual das relações entre os *tokens* do chamado, onde o tamanho de cada nó é proporcional à importância do termo (módulo do valor SHAP ou LIME, conforme o método escolhido em um seletor da interface), e a espessura das conexões indica a frequência de co-ocorrência entre os *tokens*;
- **Estatísticas**: painel que apresenta numericamente a importância individual de cada *token* calculada pelos métodos SHAP e LIME, permitindo ao analista comparar e validar as contribuições de cada termo na decisão do modelo.

A escolha por disponibilizar tanto visualizações gráficas quanto estatísticas numéricas busca garantir que analistas com diferentes perfis consigam auditar as decisões do modelo, seja por meio de padrões visuais no grafo ou por comparações quantitativas entre os valores de importância dos *tokens*.

Na lateral esquerda do sistema encontra-se a seção “Tickets recentes”, responsável por apresentar classificações realizadas anteriormente juntamente com seus respectivos níveis de confiança. Esse recurso permite consultar rapidamente chamados processados anteriormente e auxilia na análise das classificações realizadas.

Os protótipos desenvolvidos tiveram como principal objetivo verificar se os recursos de explicabilidade seriam apresentados de forma intuitiva, clara e acessível aos analistas de TI.

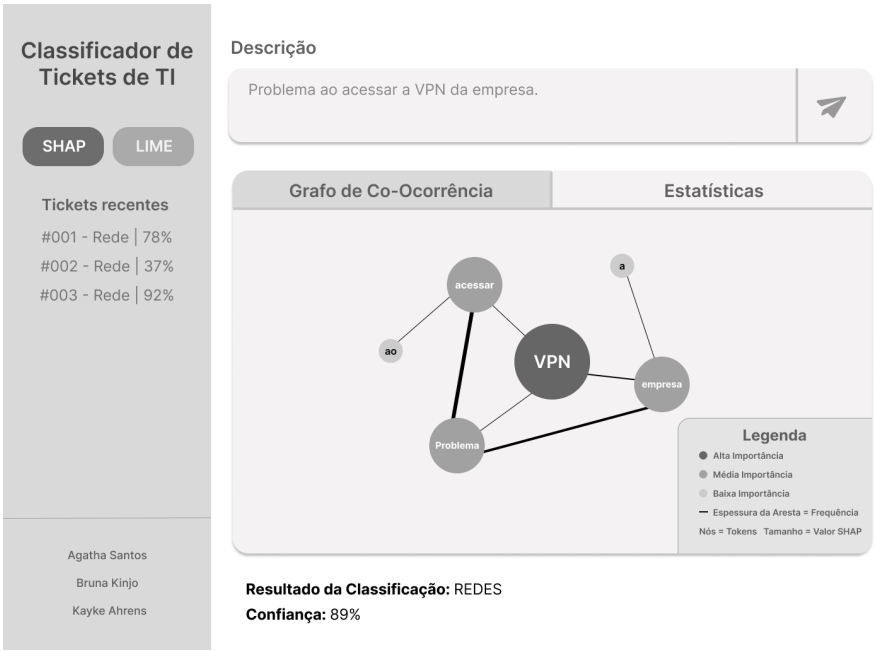


Figura 8 – Protótipo de média fidelidade - visualização do grafo de co-ocorrência



Figura 9 – Protótipo de média fidelidade - visualização estatística dos *tokens*

4.1.3 Arquitetura da solução

A arquitetura da solução é organizada em três camadas principais:

- Camada de apresentação (*front-end*);
- Camada de processamento;
- Camada de dados.

A camada de apresentação é responsável pela interação com o usuário final, disponibilizando os formulários de inserção de chamados, exibição das classificações, visualização dos grafos e acesso ao histórico de *tickets* processados.

A camada de processamento é responsável pela execução das etapas de pré-processamento textual, inferência do modelo BERTimbau, cálculo das explicações utilizando SHAP e LIME, além da construção dos grafos de co-ocorrência.

Já a camada de dados é responsável pelo armazenamento das informações relacionadas aos chamados classificados e aos resultados gerados pelo sistema.

O fluxo geral de funcionamento da aplicação segue as seguintes etapas:

1. **Entrada do modelo** — a descrição curta e a descrição completa do chamado, sem remoção de *stopwords*, são passadas como um par de textos ao tokenizador do BERTimbau, que as converte em *tokens* com seus respectivos IDs e máscaras de atenção. A remoção de *stopwords* é usada apenas na construção do grafo.

2. **Classificação com BERTimbau** — o modelo BERTimbau com *fine-tuning* processa a sequência de *tokens* e produz, a partir da representação do *token* [CLS], as probabilidades de pertencimento a cada categoria. A categoria com maior probabilidade é selecionada como predição, e o índice de confiança corresponde a essa probabilidade máxima.

3. **Cálculo de explicabilidade** — sobre o mesmo texto de entrada, são calculados os valores SHAP e as importâncias LIME. Cada *token* recebe um valor numérico que representa sua contribuição para a predição: valores positivos indicam contribuição para a classe predita, e valores negativos indicam contribuição contrária.

4. **Construção do grafo de co-ocorrência** — os *tokens* do chamado são representados como nós de um grafo $G = (V, E, W, f_{\text{shap}}, f_{\text{lime}})$. As arestas conectam pares de *tokens* que co-ocorrem dentro de uma janela deslizante de tamanho fixo, com pesos proporcionais à frequência de co-ocorrência. Os pesos dos **nós** são determinados pelos valores SHAP e LIME calculados na etapa anterior — *tokens* com maior importância, segundo o método selecionado (SHAP ou LIME), geram nós visualmente maiores na interface. O grafo é serializado em JSON, com os campos `tokens` (nós) e `arestas`, e enviado ao *frontend*.

5. **Apresentação ao analista** — a interface recebe o JSON, renderiza o grafo e exibe a categoria predita, o índice de confiança, os *tokens* mais relevantes destacados por cor e os valores SHAP e LIME comparativos.

4.2 Detalhes de implementação

4.2.1 Tecnologias e ferramentas

As tecnologias e bibliotecas listadas a seguir representam as ferramentas utilizadas para o desenvolvimento do projeto. O detalhamento por etapa está na Tabela 3.

Interface e visualização:

- *React* e *Vite*;
- *vis-network*, para a exibição do grafo;
- CSS.

Processamento e Inteligência Artificial:

- Python;
- *PyTorch* e *Transformers*, para o BERTimbau;
- *scikit-learn*, para o TF-IDF e o SVM;
- *SHAP* e *LIME*;
- *Flask*, para a comunicação entre a interface e o modelo.

Ambiente de desenvolvimento:

- Git e GitHub;
- Máquinas locais dos integrantes do grupo.

4.2.2 Funcionalidades principais

Entre as principais funcionalidades implementadas no sistema estão:

- **Funcionalidades de explicabilidade:**
  - Construção de grafos de co-ocorrência;
  - Visualização das explicações do modelo;
  - Seleção do método de explicação (SHAP ou LIME) exibido no grafo.
- **Funcionalidades de classificação e suporte:**
  - Entrada do chamado em um único campo de descrição;
  - Classificação automática de chamados de TI;

- Exibição da categoria prevista;
- Apresentação do nível de confiança da classificação;
- Histórico simplificado de *tickets* classificados.

4.2.3 Metodologia de implementação

A implementação do projeto seguiu uma abordagem iterativa, baseada em prototipação e validações incrementais.

Inicialmente foram realizados o levantamento bibliográfico, a definição dos requisitos e a elaboração dos protótipos da interface. Em seguida, foram preparados os dados, treinados os classificadores dos cenários A e B e implementados os módulos de explicabilidade (SHAP e LIME) e de construção do grafo de co-ocorrência. Por fim, esses módulos foram integrados à interface, que passou a gerar a explicação de um chamado digitado pelo *pipeline* completo.

4.3 Experimentos e cenários de teste

Os experimentos seguiram os quatro cenários da Tabela 2, todos sobre os mesmos conjuntos de treino, validação e teste descritos na Seção 4.4.2.

4.3.1 Cenários A e B: classificação

No cenário A, o *baseline* SVM com TF-IDF foi treinado no conjunto de treino e avaliado no conjunto de teste, com a descrição curta e a descrição completa concatenadas como entrada. O TF-IDF usou unigramas e bigramas, frequência mínima de documento igual a 2 e escala sublinear; o classificador foi um *LinearSVC* com $C = 1$, sem pesos de classe e semente 42. Assim como no BERTimbau, não foram removidas *stopwords*. No cenário B, o BERTimbau foi ajustado com a configuração da Seção 4.4.2. Os dois classificadores foram comparados pelas métricas da Seção 3.4.1, com destaque para o F1-macro.

4.3.2 Cenário C: explicações SHAP e LIME

As explicações foram geradas em 01/11/2026, para o modelo final, sobre uma amostra de 98 chamados do conjunto de teste: 8 acertos de cada uma das 10 classes que tiveram acertos (80 chamados) e 18 erros do modelo. A classe OUTROS não teve acertos, e por isso a amostra não tem 11×8 acertos. Foi necessário usar uma amostra porque SHAP e LIME explicam uma predição por vez, e explicar os 5.293 chamados do teste levaria cerca de 24 horas.

O LIME usou 500 amostras por explicação e semente 42. O SHAP (versão 0.54.1) foi calculado de forma exata nos campos de até 10 palavras (87 campos); acima disso, foi

estimado por amostragem, com orçamento de 500 consultas ao modelo (79 campos). A fidelidade e a estabilidade seguiram o protocolo da Seção 3.4.2; para a estabilidade, 12 chamados foram explicados 5 vezes cada. A execução completa levou cerca de 39 minutos, em CPU, em um computador Apple.

4.3.3 Cenário D: grafo de co-ocorrência

Para cada chamado explicado, construiu-se um grafo de co-ocorrência com janela deslizante de tamanho $w = 4$. As *stopwords* foram removidas apenas nessa etapa, e não da entrada do modelo. O peso de cada aresta é o número de co-ocorrências do par de palavras na janela, e o peso de cada nó é o módulo da soma dos pesos que o SHAP ou o LIME atribuiu à palavra. O grafo foi avaliado por rastreabilidade, em estudos de caso (Seção 3.4.2).

4.4 Coleta e tratamento de dados

4.4.1 Origem e autorização de uso dos dados

Os dados utilizados neste trabalho são chamados reais de suporte técnico cedidos por duas empresas do setor farmacêutico: Aché Laboratórios Farmacêuticos e Bayer S.A. Os autores solicitaram o acesso por e-mail, explicando o objetivo acadêmico do trabalho, quais informações seriam necessárias e como os registros seriam tratados.

As autorizações de uso foram concedidas pelas duas empresas por e-mail, com a condição de que os chamados fossem anonimizados antes de qualquer etapa de processamento. Essas mensagens são o registro da autorização concedida e delimitam a finalidade acadêmica do uso, e estão reproduzidas nos Anexos A e B.

A anonimização, além de atender a essa condição, alinha o trabalho ao tratamento que a Lei Geral de Proteção de Dados Pessoais (LGPD, Lei n.º 13.709/2018) dá aos dados anonimizados, que não são considerados dados pessoais para os fins da lei, salvo quando a anonimização puder ser revertida com esforços razoáveis (BRASIL, 2018). A etapa descrita na Seção 4.4.2 deixa de ser, portanto, apenas uma escolha metodológica.

4.4.2 *Dataset* e *fine-tuning*

O modelo BERTimbau, em sua versão *base*, foi ajustado com chamados reais da Bayer Brasil e da Aché, obtidos mediante autorização concedida por e-mail e anonimização dos registros. O arquivo da Bayer tinha 191.350 chamados; após a remoção de duplicatas pelo número do chamado, o descarte de chamados sem categoria ou de categorias com menos de 100 chamados e a anonimização, restaram 189.909, dos quais 46.584 estavam em português e foram mantidos por um filtro de idioma. O arquivo da Aché tinha 9.317

chamados; após o descarte das categorias com menos de 100 chamados e dos chamados sem descrição, restaram 8.043. As categorias das duas empresas foram harmonizadas em uma taxonomia única, e o conjunto combinado, com 54.627 chamados, passou ainda pelo descarte de classes com menos de 100 chamados e pelo tratamento de textos idênticos com categorias divergentes. O resultado foram 53.279 chamados (Aché: 6.949; Bayer: 46.330), em 11 classes, entre elas a classe OUTROS. A base da Bayer possui, além da descrição completa, uma descrição curta, que a base da Aché não tem; nesse caso, o campo entra vazio. Ainda há chamados com texto repetido nas bases, sobretudo na Aché, o que motivou o agrupamento descrito a seguir.

O conjunto combinado foi dividido em 42.692 chamados de treino, 5.294 de validação e 5.293 de teste (proporção aproximada de 80/10/10). A divisão usou *StratifiedGroupKFold* (8 partes para treino, 1 para validação e 1 para teste), estratificada pela categoria e agrupada pelo texto do chamado: chamados com texto idêntico ficam sempre na mesma parte.

O processo de *fine-tuning* seguiu uma abordagem supervisionada de classificação textual. A entrada do modelo é o par formado pela descrição curta e pela descrição completa, limitado a 256 *tokens*; cerca de 10,2% dos pares ultrapassam esse limite. O treino usou 3 épocas, *batch* de 16, taxa de aprendizado de 2×10^{-5} e semente 42. Para reduzir o risco de *overfitting*, fenômeno que ocorre quando o modelo aprende os dados de treino com precisão excessiva e perde a capacidade de generalizar para novos exemplos, o desempenho é acompanhado no conjunto de validação durante o treino. Não foram aplicadas técnicas de balanceamento de classes nem pesos por classe; o desbalanceamento é tratado na avaliação, com o uso do F1-macro.

5 Resultados e Análise

5.1 Apresentação dos resultados

5.1.1 Desempenho da classificação (cenários A e B)

5.1.2 Explicabilidade: SHAP e LIME (cenário C)

5.1.3 Grafo de co-ocorrência (cenário D)

5.2 Análise crítica e comparação

5.3 Discussão

6 Conclusão

6.1 Retomada do problema e dos objetivos

6.2 Contribuições para a Computação

6.3 Limitações e trabalhos futuros

Referências

ATTANASIO, G. et al. ferret: a framework for benchmarking explainers on transformers. In: *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. [S.l.]: Association for Computational Linguistics, 2023. 42

BORGES, B. R. *Análise comparativa de algoritmos de classificação de texto*. Dissertação (Trabalho de Conclusão de Curso – Sistemas de Informação) — Universidade Federal de Uberlândia, 2024. Disponível em: <https://repositorio.ufu.br/handle/123456789/43701>. 23

BRASIL. *Lei n.º 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD)*. 2018. Diário Oficial da União, Brasília, DF, 15 ago. 2018. Disponível em: <https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm>. Acesso em: 26 ago. 2026. 51

DEVLIN, J. et al. BERT: Pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the Annual Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. [s.n.], 2019. Disponível em: <https://arxiv.org/abs/1810.04805>. 19, 20, 21, 35

HARUN, N. A.; HUSPI, S. H.; IAHAD, N. A. Question classification framework for helpdesk ticketing support system using machine learning. *International Journal on Informatics and Communication Technology (IJIC)*, v. 13, n. 1, 2023. Disponível em: <https://ijic.utm.my/index.php/ijic/article/view/388>. 15

JACOVI, A.; GOLDBERG, Y. Towards faithfully interpretable NLP systems: How should we define and evaluate faithfulness? In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2020. p. 4198–4205. Disponível em: <https://aclanthology.org/2020.acl-main.386/>. 36

LUNDBERG, S. M.; LEE, S.-I. A unified approach to interpreting model predictions. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [s.n.], 2017. v. 30. Disponível em: <https://arxiv.org/abs/1705.07874>. 12, 26, 27, 30, 35, 36

MIKOLOV, T. et al. Efficient estimation of word representations in vector space. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2013. Disponível em: <https://arxiv.org/abs/1301.3781>. 18

MUSTAFA, S.; Hama Saeed, M. Empowering text classification with NLP and explainable AI for enhanced interpretability. *Journal of Electrical Systems and Information Technology*, Springer, v. 12, n. 81, 2025. Disponível em: <https://link.springer.com/article/10.1186/s43067-025-00273-2>. 26, 29

PARAMESH, S. P.; SHREEDHARA, K. S. *A Deep Learning Based IT Service Desk Ticket Classifier Using CNN*. 2022. ResearchGate. Disponível em: <https://www.researchgate.net/publication/366986777>. 16

PEREIRA, L. S. B. et al. Machine learning for classification of IT support tickets. In: *International Conference on Cyber Management and Engineering (CyMaEn)*. IEEE, 2023. Disponível em: <https://www.researchgate.net/publication/368906495>. 15, 16, 33

RAZMA, A.; JURKUS, R. AI-based classification of IT support requests in enterprise service management systems. *Systems*, MDPI, v. 14, n. 2, 2026. Disponível em: <https://doi.org/10.3390/systems14020223>. 11, 14, 18, 23, 24, 29, 33, 36

RIBEIRO, M. T.; SINGH, S.; GUESTRIN, C. "why should I trust you?": Explaining the predictions of any classifier. In: *22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016. Disponível em: <https://arxiv.org/abs/1602.04938>. 12, 24, 25, 30, 35, 36

SALIH, A. et al. A perspective on explainable artificial intelligence methods: SHAP and LIME. *Advanced Intelligent Systems*, Wiley, 2023. Disponível em: <https://arxiv.org/abs/2305.02012>. 23, 24, 26, 28, 35, 36

SOUZA, F.; NOGUEIRA, R.; LOTUFO, R. BERTimbau: Pretrained BERT models for Brazilian Portuguese. In: *Brazilian Conference on Intelligent Systems (BRACIS)*. Springer, 2020. Disponível em: <https://www.researchgate.net/publication/345395208>. 22, 35, 40

TALEBI, S.; ABDOLVAND, N. Building safer social spaces: Body shaming detection using RoBERTa, LIME and term co-occurrence graph. *International Journal of Web Research (IJWR)*, 2025. Disponível em: <https://doaj.org/article/3a3f1e1beed847909118aac97aacce05>. 32, 33

VASWANI, A. et al. Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [s.n.], 2017. v. 30. Disponível em: <https://arxiv.org/abs/1706.03762>. 11, 19, 20, 35

WAHBA, Y. M. *A Hybrid Continual Machine Learning Model for Efficient Hierarchical Classification of Domain-Specific Text in The Presence of Class Overlap (Case Study: IT Support Tickets)*. Tese (Tese de Doutorado em Ciência da Computação) — University of Western Ontario, 2023. Disponível em: <https://ir.lib.uwo.ca/etd/9192>. 15, 17, 18, 34, 37, 40

YANG, X.; CUI, Y. BERT-enhanced text graph neural network for classification. *Entropy*, MDPI, v. 23, n. 11, 2021. Disponível em: <https://www.mdpi.com/1099-4300/23/11/1536>. 32

YAO, L.; MAO, C.; LUO, Y. Graph convolutional networks for text classification. In: *AAAI Conference on Artificial Intelligence*. [s.n.], 2019. Disponível em: <https://arxiv.org/abs/1809.05679>. 30, 31, 35

YOKOTA, B. *Aprendizado de máquina baseado em Classificação hierárquica de textos aplicado ao contexto chamados de suporte*. Dissertação (MBA em Inteligência Artificial e Big Data) — Universidade de São Paulo – USP/ICMC, 2025. Disponível em: <https://bdta.abcd.usp.br/directbitstream/43ebe5d7-98d6-4357-915e-71486db593d5/Beatriz_Yokota.pdf>. 12, 16, 33, 37

ZEMP, M. *Text Classification of Service Desk Tickets*. Dissertação (Master Thesis – MAS Data Science) — Zurich University of Applied Sciences (ZHAW), 2021. Disponível em:

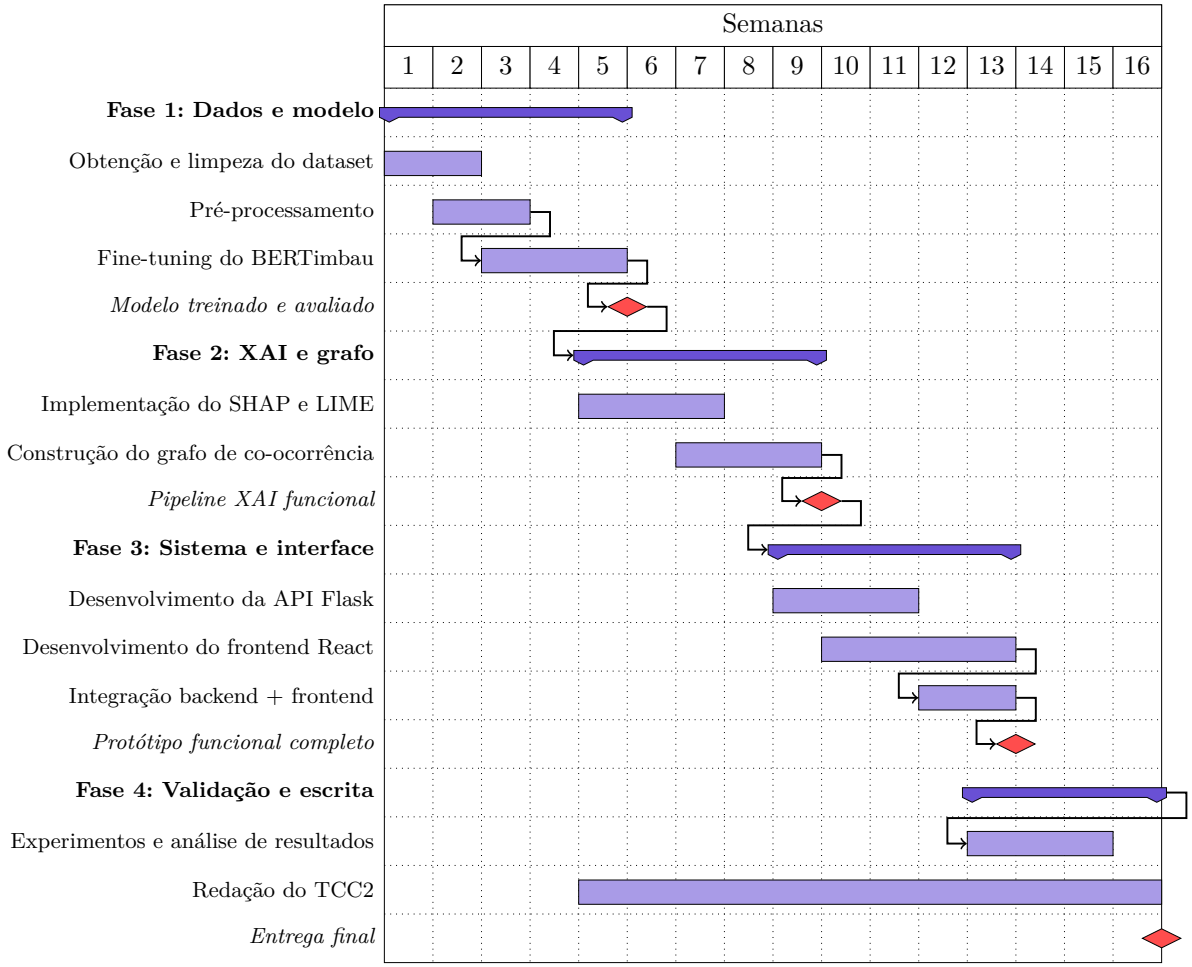
<https://www.zhaw.ch/storage/shared/upload/MAS21_Ticket_Classification_Zemp.pdf>. 11, 14, 17, 23, 24, 33, 36, 37

Apêndices

APÊNDICE A – Cronograma de atividades

O cronograma a seguir apresenta as atividades do TCC2, organizadas em quatro fases ao longo de 16 semanas. Em relação ao plano inicial, a interface passou a ser servida com *Flask*, em vez de *FastAPI*.

Figura 10 – Cronograma de atividades – Diagrama de Gantt



Fonte: Elaborado pelos autores

Anexos

ANEXO A – Autorização de uso de dados — Aché Laboratórios Farmacêuticos

Mensagem de e-mail em que a Aché Laboratórios Farmacêuticos autoriza o uso de chamados de suporte técnico para fins acadêmicos neste trabalho, mediante anonimização dos registros. Nomes, e-mails, ramais e assinaturas dos remetentes foram removidos da imagem, mantendo-se apenas o conteúdo da autorização e a identificação da empresa.

Gmail Agatha Barbosa Marinho dos Santos <agathabsantos2@gmail.com>

Fwd: Ajuda sobre Dataset do Service Desk
1 mensagem

Bruna Kinjo <brukinjo@gmail.com> 13 de agosto de 2026 às 14:19
Para: kaykebiscegli2005@gmail.com, Agatha Barbosa Marinho dos Santos <agathabsantos2@gmail.com>

--------- Mensagem encaminhada ---------
De: **Bruna Kinjo Luiz Pinto** <bruna.pinto@ache.com.br>
Data: ter., 26 de mai. de 2026 às 12:26
Assunto: ENC: Ajuda sobre Dataset do Service Desk
Para: Bruna Kinjo <brukinjo@gmail.com>

De: Batistela, Tatiana <tatiana.batistela@dxc.com>
Enviado: segunda-feira, 25 de maio de 2026 11:44
Para: Bruna Kinjo Luiz Pinto <bruna.pinto@ache.com.br>
Cc: Camila Cristina Grozinski <camila.grozinski@ache.com.br>; Patricia Bispo Das Neves <patricia.neves_terceiro@ache.com.br>
Assunto: RE: Ajuda sobre Dataset do Service Desk

! **E-MAIL EXTERNO:** Este e-mail foi enviado por alguém fora da organização. Por favor, tenha cuidado ao clicar em links ou abrir anexos.

Bruna,

Bom dia.

Aqui está a base de chamados de Abril, conforme combinamos.

Grata,

Tatiana G. Batistela
Service Delivery Manager

Mobility and Workplace Services

DXC Technology
São Bernardo do Campo – São Paulo Brazil

DXC Enterprise Technology & Innovation Partner

From: Camila Cristina Grozinski <camila.grozinski@ache.com.br>
Sent: Tuesday, May 12, 2026 9:32 PM
To: Bruna Kinjo Luiz Pinto <bruna.pinto@ache.com.br>; Batistela, Tatiana <tatiana.batistela@dxc.com>; Patricia Bispo Das Neves <patricia.neves_terceiro@ache.com.br>
Subject: RE: Ajuda sobre Dataset do Service Desk

@Batistela, Tatiana @Patricia Bispo Das Neves podem por favor apoiar?

aché mais vida para você

Camila Grozinski
Coordenadora Serviços de Suporte T.I.
Aché Laboratórios Farmacêuticos S.A.
+55 (11)2608-6770 +55 (11) 94227-2846
camila.grozinski@ache.com.br
www.ache.com.br

ANEXO B – Autorização de uso de dados — Bayer S.A.

Mensagem de e-mail em que a Bayer S.A. autoriza o uso de chamados de suporte técnico para fins acadêmicos neste trabalho, mediante anonimização dos registros. Nomes, e-mails, ramais e assinaturas dos remetentes foram removidos da imagem, mantendo-se apenas o conteúdo da autorização e a identificação da empresa.

Gmail Agatha Barbosa Marinho dos Santos <agathabsantos2@gmail.com>

Fwd: Ajuda sobre Dataset do Service Desk
1 mensagem

Bruna Kinjo <brukinjo@gmail.com> 13 de agosto de 2026 às 14:19
Para: kaykebiscegli2005@gmail.com, Agatha Barbosa Marinho dos Santos <agathabsantos2@gmail.com>

--------- Mensagem encaminhada ---------
De: **Bruna Kinjo Luiz Pinto** <bruna.pinto@ache.com.br>
Data: ter., 26 de mai. de 2026 às 12:26
Assunto: ENC: Ajuda sobre Dataset do Service Desk
Para: Bruna Kinjo <brukinjo@gmail.com>

De: Batistela, Tatiana <tatiana.batistela@dxc.com>
Enviado: segunda-feira, 25 de maio de 2026 11:44
Para: Bruna Kinjo Luiz Pinto <bruna.pinto@ache.com.br>
Cc: Camila Cristina Grozinski <camila.grozinski@ache.com.br>; Patricia Bispo Das Neves <patricia.neves_terceiro@ache.com.br>
Assunto: RE: Ajuda sobre Dataset do Service Desk

! **E-MAIL EXTERNO:** Este e-mail foi enviado por alguém fora da organização. Por favor, tenha cuidado ao clicar em links ou abrir anexos.

Bruna,

Bom dia.

Aqui está a base de chamados de Abril, conforme combinamos.

Grata,

Tatiana G. Batistela
Service Delivery Manager

Mobility and Workplace Services

DXC Technology
São Bernardo do Campo – São Paulo Brazil

DXC Enterprise Technology & Innovation Partner

From: Camila Cristina Grozinski <camila.grozinski@ache.com.br>
Sent: Tuesday, May 12, 2026 9:32 PM
To: Bruna Kinjo Luiz Pinto <bruna.pinto@ache.com.br>; Batistela, Tatiana <tatiana.batistela@dxc.com>; Patricia Bispo Das Neves <patricia.neves_terceiro@ache.com.br>
Subject: RE: Ajuda sobre Dataset do Service Desk

@Batistela, Tatiana @Patricia Bispo Das Neves podem por favor apoiar?

achê
mais vida para você

Camila Grozinski
Coordenadora Serviços de Suporte T.I.
Aché Laboratórios Farmacêuticos S.A.
+55 (11)2608-6770 +55 (11) 94227-2846
camila.grozinski@ache.com.br
www.ache.com.br